PYTHON

04

DB and Python Framework

Build and deploy a real, database-backed web application using Django — end to end.

DURATION	LEVEL	MODULE	UPDATED
20 Sessions	Intermediate	Module 4	May 2026

IN THIS MODULE

SESSION
01

60 MIN · WEB FRAMEWORKS

Introduction to Django & Setup

Learning Objectives

- Explain what a web framework is and what work Django saves you
- Describe Django's MVT architecture in your own words
- Compare Django and Flask and pick the right one for a given app
- Install Django on your machine and confirm it works
- Create a new Django project and run the development server
- Recognise the role of each file the startproject command creates

01 What Django actually is — and what work it saves you

A web framework is a pre-built toolbox for making websites. Without one, every time you build a site you would also have to write — from scratch — the code that reads incoming web requests, the code that talks to a database, the code that handles user login, the code that protects against attacks, and many more boring-but-essential parts. A framework gives you all of that ready-made, so you can focus on what makes your app different from the rest.

Django is one such framework, written in Python. It was created in 2003 inside a small American newspaper office where reporters needed to publish new pages every single day. The team needed something that let them build fast, change fast, and not break. That origin shows in Django's personality even today: it gives you a lot of things by default — an admin panel, a login system, form handling, security protections — so you spend less time on plumbing and more time on the actual website.

> **CONCEPT — A FRAMEWORK IS NOT A LIBRARY**
>
> A library is something you call when you need it — like requests in Python, you import it and use it. A framework is the opposite: it calls you. Django decides the shape of your project, where files go, how requests flow. You fill in the pieces. Think of building a house: a library is a hammer you pick up when needed; a framework is the entire floor plan that tells you where the kitchen goes.

When students in Ahmedabad learn Django, they are learning the same framework that runs apps with hundreds of millions of users worldwide. The same Django that handles a small college event website can — with the right architecture — handle a platform with millions of requests an hour. That's not common in the framework world.

02 MVT — how Django splits up your code

Django is built around a pattern called MVT — Model, View, Template. The idea is simple: a web app does three different kinds of work, and Django wants you to keep them in three different places. This separation is what keeps a Django project understandable even when it grows to thousands of files.

CONCEPT — MVT IN ONE PARAGRAPH

Model = what the data looks like (a Student has a name, a roll number, a course). View = the logic that runs when someone visits a URL (fetch students from the database, decide what to show). Template = the HTML page the user actually sees (a list of student cards). When a request comes in, Django takes the URL, picks the right View, the View talks to the Model to get data, and then hands that data to a Template which becomes the final web page. Three jobs, three layers, no mixing.

An everyday way to picture this: imagine a college canteen counter. The Model is the kitchen — where the actual food (data) is stored and cooked. The View is the cashier — they take your order, walk to the kitchen, collect what you asked for, and arrange it on a tray. The Template is the tray and plates — the way the food is presented to you. The customer never sees the kitchen directly, and the kitchen never decides how the food looks on the plate. Each role stays separate.

“MVT is not about Django being fancy. It is about future-you, six months from now, being able to find your own code.”

03 Django or Flask — which one, when?

Flask is another popular Python web framework. The two are often compared because they take opposite philosophies. Django is opinionated — it comes with batteries included and expects you to follow its way of doing things. Flask is minimalist — it gives you the bare essentials and lets you assemble the rest yourself, picking your own database tool, your own form library, your own auth system.

Django shines when…

You are building something with many moving parts — user accounts, an admin panel, a database with related tables, file uploads, email, scheduled jobs. A college portal where students log in, faculty post assignments, and parents check attendance is a classic Django shape. The defaults Django gives you cover 80 percent of what you need on day one.

Flask shines when…

You are building something small and focused — a single API endpoint that takes a phone number and sends an OTP, a tiny dashboard that pulls one CSV and shows a chart, a microservice that does one thing inside a larger system. You do not need a whole admin panel; you would rather pick three small libraries than inherit Django's full machinery.

There is no "better" framework. There is only the better fit for the job. The same engineer often uses both — Django for the main product, Flask for a small internal tool that runs alongside it.

04 Installing Django and creating a project

Django is just a Python package, so the install is one command. The standard tool for installing Python packages is pip — it ships with Python itself. Once you run pip install, Django becomes available everywhere on your system: any folder, any project. Verifying the install is its own small skill — it is how you confirm pip didn't fail silently, and how you confirm your terminal can find Django when you call it.

```
pip install django
```
Downloads Django from PyPI and installs it into your active Python environment.

```
django-admin --version
```
Prints the installed Django version. If you see a number, pip worked AND your PATH is set up correctly.

```
django-admin startproject campus_events_project
```
Creates a new project folder named campus_events_project with all the starter files.

```
cd campus_events_project
```
Move into the new folder before running anything else.

```
python manage.py runserver
```
Starts Django's built-in development web server on port 8000.

TRY THIS LIVE

Open a terminal and run `django-admin --version` BEFORE you install Django. Notice what error message you get. Then install Django and run the same command again. The difference is small but worth seeing — the first time you see "command not found", you understand for the rest of your career what PATH problems look like.

PRO TIP — THE DEV SERVER IS NOT THE REAL SERVER

When you run `python manage.py runserver` and open 127.0.0.1:8000, that's Django's built-in development server. It is convenient and fast to restart, but it is single-threaded and not safe for the public internet. In real production — when your app is live for actual users — you replace it with Gunicorn or uWSGI behind Nginx. The dev server is a learning tool, not a deployment tool. Knowing this on day one saves confusion six months later.

05 The shape of a Django project — what each file is for

When you run `startproject`, Django creates a small set of files for you. They look intimidating at first but each one has a clear and narrow job. Understanding what each file is responsible for is the difference between editing the right place and breaking your project by changing the wrong line.

File	What it does	How often you edit it

File	What it does	How often you edit it
manage.py	The command-line door into your project. You run things like runserver, makemigrations, createsuperuser through it. It almost never needs editing.	Rarely — only for advanced setups
settings.py	Your project's control panel. Database connection, installed apps, secret key, time zone, allowed hosts, static files — all configured here.	Often — especially early on
urls.py	The map between URL paths and views. When a request comes in, Django looks here first to figure out which view should handle it.	Often — every new page adds a route
wsgi.py	The bridge between Django and a traditional production web server like Gunicorn. Used when deploying.	Rarely — usually just on deployment day
asgi.py	Same idea as wsgi, but for the newer async world — needed if you add WebSockets or real-time features later.	Rarely — only if you go async

CONCEPT — settings.py IS THE FIRST FILE YOU SHOULD READ

Before writing a single line of your own code in a new Django project, scroll through settings.py once. Every Django decision your project will make — which database, which timezone, which apps are active — is declared here. Reading it is how you teach yourself what Django thinks a project is.

CASE STUDY — INSTAGRAM, BUILT ON DJANGO

Context — Instagram started in October 2010 in San Francisco with just two founders, Kevin Systrom and Mike Krieger. They had no infrastructure team, no big-tech budget, and a deadline measured in weeks.

Challenge — They needed a backend they could build fast — but it also had to survive whatever growth came next. They had no idea their app would cross 25,000 signups on day one and a million users within two months.

What they did — They picked Django and Python on top of AWS. Django gave them the user model, the admin panel, the ORM and the request handling out of the box, so the two of them could focus on the photo feed, the filters and the social graph rather than re-inventing login pages.

Result — Today Instagram runs what its own engineering team has publicly called "the world's largest deployment of the Django web framework" — serving billions of users on essentially the same framework that two founders set up in 2010. The codebase has grown to several million lines, but the MVT shape is still there.

Takeaway — Django was originally built for a small newspaper. The same framework now runs one of the biggest social apps on earth. The lesson is not "Django scales magically" — it doesn't, the Instagram team has spent years tuning it. The lesson is: pick the framework that helps you ship today, and worry about scale when you have the problem.

Source: Instagram Engineering, "Web Service Efficiency at Instagram with Python" (2016), instagram-engineering.com

WATCH — VIDEO REFERENCE (ENGLISH)

Channel — NPTEL — IIT Kharagpur

Watch — Internet Technology by Prof. Indranil Sengupta — start with Lecture 1 (Introduction to the Internet) and Lecture 2 (Review of Network Technologies). NPTEL does not have a dedicated Django course, but Django sits on top of HTTP and the client-server model — and this is the official IIT lecture series that teaches those foundations.

Link — https://nptel.ac.in/courses/106105084

Always link the official channel — beware fan-made or re-upload copies.

AI PROMPT — ASK CHATGPT / GEMINI / CLAUDE

Act as a friendly Python mentor. Explain to me in simple Hinglish what the MVT pattern in Django is, using the example of a college canteen counter (kitchen, cashier, tray). Then ask me one question to check if I understood. Keep your answer under 150 words.

WHERE TO GO NEXT

Now that you can install Django and create a project, the next session is about apps inside a project — and that is where the real building begins.

Explore deeper — The official Django tutorial at docs.djangoproject.com — it walks you through building a small poll app. Don't copy-paste; read each step and predict what the next command will do before running it.

Try on your own — Create a second Django project on your machine with a different name. Compare the two folders side by side. Are they identical? Why does Django regenerate the same files every time?

Connect the dots — Session 2 introduces Django apps — the smaller units of code inside a project. Today you made the building; next session you start adding rooms to it.

Going further — Read the Instagram Engineering blog post mentioned in the case study. You won't understand every line yet, but notice how often the words "Django", "Python" and "deploy" appear. That's a hint about what real backend work looks like.

USING THIS HANDOUT FOR YOUR PRACTICE TASKS

Your four tasks for this session are in the assignments system. This handout is your thinking partner — it explains the ideas you'll need, but the answers are yours to find. Each pointer below tells you which section to

revisit and what question to ask yourself.

Task 1.1 — Section 4 explains why pip is the standard install tool and shows the verify command. After your install, ask yourself: what does `django-admin --version` actually prove? Does a version number on screen confirm just that Django is installed — or also that your terminal can find it? What would happen if you opened a brand new terminal tab and ran the same command?

Task 1.2 — Section 4 walks through `startproject` and `runserver` using a different project name. Once your dev server is running on port 8000, ask yourself: what does the URL 127.0.0.1 actually point to? Why isn't Django visible on your phone over the same Wi-Fi by default? The Pro Tip callout in Section 4 hints at part of the answer.

Task 1.3 — Section 5 has a table describing each file. Don't just copy the table — open each file in your own project first and READ a few lines before checking the table or asking an AI. For each file, ask: "if I deleted this, what would break?" That question, more than any documentation, teaches you what the file is really for.

Task 1.4 — Section 3 and the comparison block explain the Django-vs-Flask philosophy. To write your 3-4 line comparison: pick an app you use every day, then list five features it has — login, search, profile, recommendations, payments. Count how many of those Django would give you for free, then how many a tiny Flask app would. The bigger the gap, the clearer your answer about which framework fits.

“A framework's job is not to impress you. It is to remove the boring work so you can do the interesting work.”

SESSION
02

60 MIN · WEB FRAMEWORKS

Apps, Settings & URL Routing

Learning Objectives

- Explain the difference between a Django project and a Django app using your own real-world analogy
- Create a new Django app using the startapp command and identify the purpose of every file it generates
- Register an app in INSTALLED_APPS and explain what Django breaks silently when you forget
- Describe the roles of settings.py, urls.py, and wsgi.py inside a running Django project
- Write a function-based view that returns an HttpResponse and wire it to a URL path
- Trace a browser request step by step — from the address bar through urls.py to the view to the response

01 Project vs App — the campus and its buildings

A Django project is like a college campus. The campus is one entity — one address, one administration, one electricity bill. But inside the campus there are many separate buildings: the library, the hostel, the canteen, the sports department, the examination office. Each building handles its own work, has its own staff, and can largely function on its own — yet they all share the same campus infrastructure. In Django, your project is the campus, and each app is one of those buildings.

More precisely: a project is a single Django codebase. It has one settings.py, one root urls.py, one database connection, and one manage.py entry point. Apps are the smaller, focused modules that live inside that project. A student information system built for a Surat college might contain an accounts app (handles login and signup), a timetable app (manages class schedules), and a results app (stores and displays marks). Each app lives in its own folder and manages its own piece of the problem — while sharing the project's database connection, settings, and admin interface.

> **CONCEPT — APPS ARE DESIGNED TO BE REUSABLE**
>
> Django apps are meant to be self-contained enough to be picked up from one project and dropped into another. An accounts app you write for a library management system should need very little change to work inside a hospital appointment system. This is why popular third-party packages exist: django-allauth handles social login, django-crispy-forms handles form rendering, and both work simply by being added to INSTALLED_APPS in any project. The apps you build today can become the building blocks of your next project tomorrow.

02 Creating an app and exploring what's inside

You create an app with one command. Django generates a folder with all the starter files the app will need. Before you check any documentation, open that folder and look at every file. Ask yourself what job each one might have — the names are hints. This habit of reading before looking things up trains the kind of intuition that experienced developers have.

```
python manage.py startapp library
ls library/
```

Creates a new app folder named 'library' inside your project directory. Lists all files Django generated. On Windows, use: dir library

File	What it is for	How often you edit it
views.py	Where you write the Python functions (or classes) that handle web requests and return responses. Every new page starts here.	All the time
models.py	Where you describe the shape of your data — the fields a Book has, what a Member record looks like. Django turns these into database tables.	Whenever you add or change a data structure
urls.py	App-level URL patterns. Django does NOT create this file for you — you create it when you are ready to add routing for this app.	When adding new pages to this app
admin.py	Register this app's models so they appear in Django's built-in admin panel at /admin/.	When you want staff to manage data via admin
apps.py	Metadata about the app — its full Python path and any startup configuration. Rarely needs editing.	Rarely — mostly left as generated
migrations/	Auto-generated files that track every change to your models over time. Django creates these for you via makemigrations.	Never manually
tests.py	Where you write automated tests to verify your views and models behave correctly.	Grows as the project matures

PRO TIP — DJANGO DOES NOT CREATE urls.py FOR YOU

When you run startapp, Django creates views.py, models.py, admin.py, and tests.py — but not urls.py inside the app. You create that file yourself when the app is ready to receive requests. This is intentional: Django trusts you to design your URL structure consciously, not auto-generate it. When you are ready, create a new file named urls.py inside the app folder and start adding patterns.

03 Registering your app — the INSTALLED_APPS list

Creating an app folder is not enough. Django does not scan your project directory for new folders and automatically include them. You have to register your app. Think of joining a new college: you might show up on campus, attend classes, and sit in the canteen — but you are not officially a student until the admin office adds your name to the register. In Django, that register is the INSTALLED_APPS list inside settings.py.

The settings.py file is your project's control panel. It holds every top-level decision Django makes: which database to connect to, which timezone to use, where to find static files, what the secret key is for security, and — critically — which apps are active. Every line in that file is a configuration choice that affects the entire project, so reading through it once at the start of a new project is a good habit. INSTALLED_APPS is just one section of it, but it is the section that affects the most things.

CONCEPT — THREE THINGS INSTALLED_APPS CONTROLS AT ONCE

Django uses INSTALLED_APPS for three separate purposes that are easy to forget are connected. First: migrations only run for registered apps — if your app is not listed, makemigrations ignores your models completely. Second: the template engine searches inside registered app folders for template files. Third: the admin panel only shows models from registered apps. A missing entry in INSTALLED_APPS breaks all three simultaneously — with no error message, your features just silently do not appear.

```
# In settings.py — find the INSTALLED_APPS list near the top
```

Open settings.py. INSTALLED_APPS is usually within the first 25 lines.

```
INSTALLED_APPS = [
```

Start of the list.

```
    'django.contrib.admin',
```

Django's built-in admin panel — already registered.

```
    'django.contrib.auth',
```

Django's built-in authentication system — already registered.

```
    'django.contrib.contenttypes',
```

Internal Django machinery for content type tracking.

```
    # ... other defaults ...
```

Django ships with six built-in apps. All are already listed.

```
    'library',
```

Add your app's name here as a plain string. Save the file after.

```
]
```

Run the server again — no errors means the app is registered correctly.

WARNING — MISSING INSTALLED_APPS IS THE #1 MISTAKE IN THIS SESSION

If your model refuses to migrate, your admin does not show a model, or your template silently fails to load — check INSTALLED_APPS in settings.py before anything else. It takes ten seconds to look and saves twenty minutes of confused debugging. This is so common that experienced Django developers check it first when helping someone whose 'app is not working'.

04 URL routing — how Django reads the address bar

Every time a browser sends a request to your Django server, Django's first question is: which view function should handle this? It answers that question by reading the URL path — everything after the domain name. This routing system works like the security gate of a large college: the guard looks at where you say you are going and directs you to the right building. The routing rules are written in urls.py.

The routing file contains a list called urlpatterns. Each item in this list maps a URL path to a view function. Django reads the patterns from top to bottom and picks the first one that matches the incoming URL. If nothing matches, Django returns a 404 Not Found response — its way of saying 'nobody handles that address'. If two patterns could match, only the first one wins, so order matters.

> **CONCEPT — ANATOMY OF A URL PATTERN**
>
> A URL pattern has three parts. The path string — like 'books/' or 'books/<int:book_id>/' — is what Django matches against the URL. The view function — like views.book_list — is what Django calls when the path matches. The optional name — like name='book_list' — is a label you use to generate that URL inside templates without hardcoding the path string. The angle-bracket syntax <int:book_id> is a URL parameter: Django captures that portion of the URL and passes it to your view as a named argument automatically.

```
# library/urls.py — create this file yourself inside the app folder
    Django does not auto-generate this file. Create it.
from django.urls import path                    Import Django's path function for defining URL patterns.
from . import views                             Import the views module from this same app folder (dot
                                                = current package).
urlpatterns = [                                 The list Django reads to match incoming URLs.
path('books/', views.book_list, name='book_list'),
    Maps /books/ to the book_list view function.
path('books/<int:book_id>/', views.book_detail, name='book_detail'),
    Captures a number from the URL and passes it to book_detail as book_id.
]                                               End of urlpatterns.
```

As projects grow, keeping all URL patterns in a single root file becomes unmanageable. Django solves this with the include() function: the project's root urls.py handles the 'which app?' decision, and each app's own urls.py handles the 'which page in that app?' decision. This keeps routing modular — the same idea as keeping code modular. When a request comes in, Django reads the root urls.py first, finds the matching include(), and then reads that app's urls.py to find the final view.

```
# projectname/urls.py — the root file, connects all apps
    Edit the root urls.py that django-admin created for the project.
from django.contrib import admin                Admin import is already there by default.
from django.urls import path, include           Add 'include' to the import — it's not there by default.
urlpatterns = [                                 Root URL patterns.
```

```
    path('admin/', admin.site.urls),

    path('', include('library.urls')),

]
```

Django admin panel at /admin/ — already present by default.
Delegate all URLs (empty prefix) to the library app's urls.py.
Requests for /books/ now flow into library/urls.py to find the final view.

Root urls.py (project level)

Handles big-picture routing: 'anything under admin/ goes to the admin panel; anything under library/ goes to the library app; anything under accounts/ goes to accounts.' One or two lines per app, then hands control downward using include(). Think of it as the campus directory board at the main gate — it tells you which building to go to, not which room.

App urls.py (app level)

Handles fine-grained routing inside one app: 'books/ shows the list; books/<id>/ shows one book; books/add/ shows the add form.' All patterns are specific to this app's functionality. You create this file manually — Django's startapp does not generate it. Think of it as the floor directory inside a building — it tells you exactly which room you need.

CONCEPT — WHERE wsgi.py FITS IN THE REQUEST JOURNEY

You learned in Session 1 that wsgi.py is the bridge between Django and a production web server. In the request lifecycle, it is the very first door a request passes through: a production server like Gunicorn calls wsgi.py, which hands the request to Django. Django's middleware then runs, the URL dispatcher reads urls.py to find the right view, the view runs and returns a response, and wsgi.py hands that response back to the production server. You do not write code inside wsgi.py, but knowing where it sits in the chain explains why it is the right file to point a production server at.

05 Views and HttpResponse — writing a page that answers back

A view is a Python function. It receives one mandatory argument — the request object, which contains everything about what the browser sent: the URL, any form data, cookies, the logged-in user's identity, and more. The view must return a response object — containing what the browser should display. The simplest possible response is HttpResponse: a text string sent directly back to the browser. That's it. One function in, one response out.

```
# library/views.py

from django.http import HttpResponse

def book_list(request):

return HttpResponse('TOPS Library — 437 books available')
```

Open the views.py that Django generated in your app folder.
Import HttpResponse from Django's http module.

A view is just a function. It always takes 'request' as the first argument.

Return a plain text response. This is the minimum a view must do.

```
def book_detail(request, book_id):                A view for a single book. book_id is captured from the URL pattern.
    message = f'Showing details for Book ID: {book_id}'
        Use the URL parameter in your response.
    return HttpResponse(message)                  Return the personalised response.
```

TRY THIS LIVE

Write a view in your app that returns the text 'Welcome to TOPS — your first Django page is working.' Hook it to any URL path you choose. Start the server, open a browser, and hit that URL. Notice what you built in about five lines of Python: a URL that exists, a server that responds, a message you chose. The text is plain for now — no HTML, no styling — but this is the exact same mechanism behind every web page you have ever visited. A function received your request. A function returned a response.

HttpResponse is the foundation, but Django gives you several more specialised response classes for common cases. JsonResponse sends a Python dictionary as JSON — used when building APIs. HttpResponseRedirect sends the browser to a different URL — used after a form is submitted successfully. Http404 raises a Not Found error — used when a database lookup finds nothing. All of these inherit from HttpResponse and follow the same pattern: import, instantiate, return. Once you understand the base, the others are just shortcuts.

“A view is the simplest idea in web development: a function that receives a request and returns a response. Every web page on earth follows this pattern.”

CASE STUDY — DISQUS: ONE DJANGO APP, FOUR MILLION WEBSITES

Context — Disqus is a San Francisco-based company that built the comment-section widget you see embedded at the bottom of news articles and blogs worldwide. At their peak, their comment system appeared on over four million websites and served more than 17 billion page views per month.

Challenge — A comment engine that needs to work on four million completely different websites must be a self-contained, portable unit. It cannot depend on the host site's database structure, its URL patterns, or its user authentication. Disqus needed their code to be as modular as a plug-in — dropped in anywhere, just works.

What they did — They built their comment engine as a standalone Django app. The comments app had its own models (Comment, Thread, Author), its own views (post a comment, list comments, moderate), its own URL patterns, and its own template fragments. Because it was a genuine Django app, different engineering teams could work on billing, publisher tools, and the comment engine independently — each in its own app folder — without interfering with each other. The comment app itself could be embedded as a JavaScript widget into any external site.

Result — At peak load, Disqus's comment app processed billions of requests per month. Their engineering team credited Django's modular app architecture for making it possible for a small team to build and maintain a system used by millions of publishers. Features could be shipped to one app without touching another.

Takeaway — The apps you create today — even a small library app with two views — follow the exact structural principle that Disqus used at billions of requests per month. The concept scales. The habit of thinking in apps instead of files is one of the most valuable things you build this module.

Source: Disqus Engineering Blog and PyCon US technical talks by Disqus engineers (2013–2015); disqus.com/about/blog

WATCH — VIDEO REFERENCE (ENGLISH)

Channel — NPTEL — IIT Madras

Watch — Programming, Data Structures And Algorithms using Python by Prof. Madhavan Mukund (IIT Madras) — the Week 2 lectures cover how Python organises code into modules and packages. This is the direct foundation for understanding Django apps: a Django app is a Python package, and Python's import system is exactly what Django uses to load your views, models, and URLs when it reads the INSTALLED_APPS list at startup.

Link — https://nptel.ac.in/courses/106106145

AI PROMPT — ASK CHATGPT / GEMINI / CLAUDE

Act as a Django tutor. I am building a college portal in Django for a Gujarat university. Explain to me in simple Hinglish: (1) why I should create separate apps for students, attendance, and results — instead of putting all my code in one views.py file — and (2) what happens step by step when I type http://127.0.0.1:8000/attendance/ in my browser, from the moment I press Enter to the moment the page appears. Use a GTU or LDCE-style example. Keep your answer under 200 words.

WHERE TO GO NEXT

You can now create apps, register them, route URLs, and write views that respond to requests. The next session adds the HTML layer — templates — so your views can return fully designed web pages instead of plain text strings.

Explore deeper — Django's official documentation on the URL dispatcher at docs.djangoproject.com/en/stable/topics/http/urls/ — specifically the section titled 'How Django processes a request'. Read it after completing your tasks; the documentation will make far more sense once you have done the steps hands-on.

Try on your own — Create a project with two completely separate apps — one for books, one for members. Give each app one URL and one view that returns its name. Then ask yourself: what would a third app in the same project be? The moment you start thinking in terms of apps rather than files, you are thinking like a Django architect.

Connect the dots — Session 3 replaces the plain HttpResponse text string with a full HTML template. The URL routing and view functions you set up today remain exactly the same — only what the view returns changes. Nothing you built today gets thrown away.

Going further — Read about URL namespacing in Django — the app_name attribute you add to an app's urls.py. This prevents URL name collisions when two apps both define a view called 'home'. Understanding

namespacing now saves a confusing debugging session later when your project has five apps.

USING THIS HANDOUT FOR YOUR PRACTICE TASKS

Your five tasks for this session are in the assignments system. This handout is your thinking partner — it explains the ideas you will need, but the answers are yours to find. Each pointer below tells you which section to revisit and what question to ask yourself before writing any code.

Task 2.1 — Section 2 walks through the startapp command and shows every file it creates, using a library app as the example. Once your app folder appears, open each file before looking at any documentation. Ask yourself: why does Django give you views.py and models.py on day one, even though you have not written any views or models yet? What is Django assuming you will need as the app grows? The table in Section 2 will help you check your own answers.

Task 2.2 — Section 3 explains why INSTALLED_APPS is not optional and lists exactly what Django breaks silently when it is missing. Before adding your app, look at the other apps already in the list — they are all Django's own built-in apps. Ask: if you ran makemigrations right now, before registering your app, what would you expect to happen? Try it and see if you were right. Then register the app and try again. Notice the difference.

Task 2.3 — Section 5 shows the pattern every view function follows. Read the first paragraph carefully — it describes two things every view must do. Before writing your view, ask yourself: what is the job of the 'request' argument, even if you are not using it yet in this simple view? What would happen if you forgot the return statement? The code box in Section 5 has a working example using a completely different scenario — follow the structure, not the exact words.

Task 2.4 — Section 4 covers both the project-level and app-level urls.py, and shows the include() pattern that connects them. When wiring your URL path to your view, ask: should this URL pattern live in the project's root urls.py or in your app's own urls.py? What is the reason for each? For a single-app project, both technically work — but one is the better habit. The comparison block in Section 4 will help you decide.

Task 2.5 — After changing your view's response text, pay close attention to what happens next: did Django's development server pick up the change automatically, or did you have to restart it manually? Ask yourself why Django behaves this way. This behaviour is deliberate, and understanding it will save you time throughout the rest of this module whenever you wonder 'why is my change not showing up?'

SESSION
03

60 MIN · FRONT-END LAYER

Templates & Static Files

Learning Objectives

- Explain how Django separates presentation from logic using the template engine
- Create a base.html layout using {% block %} and {% endblock %} tags
- Extend base.html into child templates using {% extends %}
- Use {{ variables }} and {% tags %} correctly inside any template
- Configure STATICFILES_DIRS and serve CSS, JS, and images using {% static %}
- Build a Bootstrap-styled base layout by loading Bootstrap via CDN

01 How Django's Template Engine Works

In Django, HTML and Python never live in the same file. A view collects data and passes it to a separate .html template — the template handles only display. This clean split is how large teams at companies like Naukri.com and ClearTax let backend developers and frontend designers work on the same project simultaneously, without stepping on each other.

> **CONCEPT — Django Template Language (DTL)**
>
> DTL is Django's lightweight HTML-plus-variables system. It has two building blocks: {{ variable }} to print a value passed from the view, and {% tag %} to run logic such as loops and conditions. Think of it as HTML with a few carefully chosen superpowers — no full Python allowed, by design.

02 Template Inheritance — base.html

Every Django project has repeated elements — a navigation bar, a footer, and a common <head> block. Instead of pasting these into every page, you write them once in base.html and reuse them everywhere using two template tags: {% extends %} and {% block %}. Changing the nav in one file updates every page instantly.

`{% extends 'base.html' %}`	Must be the first line — tells Django this template inherits from base.html
`{% block content %}{% endblock %}`	Marks a replaceable zone in the parent; child templates fill this in
`{% block title %}Doctor Finder{% endblock %}`	Override the page <title> in each child template individually
`{% include 'partials/navbar.html' %}`	Pulls in a reusable partial template — useful for complex components

TRY THIS LIVE

Create templates/base.html with a nav bar and a {% block content %}{% endblock %} placeholder. Create templates/index.html that extends it and fills the block with one heading. Run the server and confirm the nav appears on the page without being written a second time.

03 Template Tags & Variables

Syntax	Type	What It Does
{{ doctor.name }}	Variable	Prints the value of doctor.name from the context dict
{% if user.is_authenticated %}	Tag	Runs a conditional block — shows/hides based on a condition
{% for doc in doctors %}	Tag	Loops over a queryset or list passed from the view
{% url 'home' %}	Tag	Generates a URL from a named route — never hardcode paths
{{ doctor.name|upper }}	Filter	Transforms output — upper, lower, date, truncatechars, default
{% csrf_token %}	Tag	Security token — mandatory inside every POST form in Django

PRO TIP — Keep Templates Dumb

Never do calculations or database calls inside a template. Move all logic to the view, pass only the final result as context. Templates are for display only — this keeps them readable, testable, and easy to hand off to a designer.

04 Static Files — CSS, JS & Images

Static files are assets that do not change per user: stylesheets, JavaScript files, logos, and fonts. Django requires explicit setup to serve them even in development. Three steps: configure STATICFILES_DIRS in settings.py, add the {% load static %} tag at the top of your template, then reference files using {% static 'path' %}.

```
STATIC_URL = '/static/'
STATICFILES_DIRS = [BASE_DIR / 'static']
```

URL prefix for all static files — add to settings.py

Tells Django where your project-level static folder lives

```
{% load static %}
```
Must appear at the top of any template that uses {% static %}

```
<link href="{% static 'css/style.css' %}" rel="stylesheet">
```
Generates the correct absolute URL for your stylesheet

```
<img src="{% static 'images/logo.png' %}" alt="Logo">
```
Same pattern works for images and JS files

TRY THIS LIVE

Create a static/css/style.css file that changes the navbar background colour. Link it in base.html using {% static %}. Reload the page and confirm your CSS is applied. If it is not loading, check that django.contrib.staticfiles is in INSTALLED_APPS.

05 Building the Base Layout with Bootstrap

Bootstrap via CDN takes one line in base.html and gives you a fully responsive grid, pre-styled buttons, cards, and a mobile-ready navbar. For the Doctor Finder project, this means a professional UI without writing custom CSS from scratch — the same approach used by early-stage startups across Gujarat to ship fast without a dedicated designer.

```
<link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css"
rel="stylesheet">
```
Paste inside <head> in base.html — loads Bootstrap 5 CSS

```
<script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bundle.min.js"></s
cript>
```
Paste before </body> — loads Bootstrap JS with Popper included

```
<div class="container">{% block content %}{% endblock %}</div>
```
Wrap your content block in a Bootstrap container for auto margins

CASE STUDY — ClearTax (Clear)

Context — Clear (formerly ClearTax) is India's largest tax and compliance platform, used by businesses and CAs across Gujarat for GST filing, ITR, and invoice management.

Challenge — Their web application had dozens of form pages — tax filing wizards, invoice generators, compliance dashboards — all built and maintained by a growing team. Keeping a consistent header, sidebar, and footer across every page was causing constant copy-paste bugs.

What they did — They built one master base.html using Django's template inheritance system and extended it across every page. A single nav update propagated everywhere instantly — no file-by-file edits.

Result — Frontend updates and rebrand changes became predictable and fast. New pages took minutes instead of hours because developers only wrote the unique content block, not the full layout.

Takeaway — Template inheritance is not a nice-to-have — at product scale, it is what keeps your frontend from becoming fifty different codebases.

Source: Clear Engineering — engineering.clear.in (Django stack publicly documented)

WHERE TO GO NEXT

You now control how every page in your Django project looks and shares structure.

Explore deeper — Django Template Language full reference — docs.djangoproject.com/en/stable/ref/templates/language/

Try on your own — Add a footer block to base.html with your project name and a WhatsApp contact link. Confirm it appears on every page with zero repeated code.

Connect the dots — Session 04 builds the data layer using models. Session 03's templates will soon display that data — the view passes the queryset and the template renders it.

Going further — Read about Jinja2 as an alternate Django template backend — faster rendering, more Pythonic syntax, used in high-traffic Django deployments.

AI PROMPT — ASK CHATGPT / GEMINI / CLAUDE

Act as a Django teacher. Explain template inheritance in simple Hinglish using a real-life analogy — like a letterhead or stamp. Show a minimal base.html with one {% block %} and a child template that extends it. Keep it under 150 words.

SESSION
04

60 MIN · DATA LAYER
Django Models & Database Connectivity

Learning Objectives

- Define what a Django model is and how it maps to a database table
- Choose the correct field type for each attribute in a model
- Connect a Django project to a MySQL database using pymysql
- Run makemigrations and migrate to create and update tables
- Perform basic ORM operations — create, read, and filter records

01 What is a Django Model?

A model is a Python class that defines the structure of one database table. Each attribute in the class becomes a column. Django handles all the SQL — you never write CREATE TABLE, INSERT, or SELECT manually. Every major Indian product built on Django, from Practo to Instamojo, stores its entire data layer this way.

CONCEPT — ORM (Object-Relational Mapper)

An ORM translates Python code into SQL automatically. When you write Doctor.objects.filter(city='Ahmedabad'), Django generates SELECT * FROM accounts_doctor WHERE city = 'Ahmedabad' and returns the result as Python objects. You work in Python; Django speaks to the database.

```
from django.db import models                          Import Django's model base class — always at the top of models.py

class Doctor(models.Model):                           Define a model — Django will create a table named appname_doctor

    name = models.CharField(max_length=100)           Short text field — up to 100 characters
specialization = models.CharField(max_length=100)
   Another short text column
    city = models.CharField(max_length=50)            City column — useful for location filtering
    phone = models.CharField(max_length=15)           Store phone as text, not integer — preserves leading zeros

    def __str__(self):                                Controls how Django Admin and shell display this object

        return self.name                              Returns the doctor's name instead of 'Doctor object (1)'
```

02 Choosing the Right Field Type

Field Type	Stores	Example Use
CharField(max_length=n)	Short text	Name, city, phone, specialization
TextField()	Long text	Doctor bio, address, description
IntegerField()	Whole numbers	Age, experience in years, fee amount
FloatField()	Decimal numbers	Rating (4.5), consultation fee
EmailField()	Validated email address	doctor@example.com
BooleanField()	True / False	is_available, is_verified
DateField()	Date only (YYYY-MM-DD)	Date of joining, date of birth
DateTimeField()	Date + time	Appointment slot, created_at
ImageField()	File path to an uploaded image	Profile photo, clinic photo
ForeignKey()	Link to another model (many-to-one)	Doctor linked to a Specialization

PRO TIP — CharField vs TextField

Use CharField for anything with a known max length (name, city, phone). Use TextField only when the content is long and unpredictable (a bio or description). Choosing TextField for short data wastes database storage and slows index lookups.

03 Connecting Django to MySQL

Django ships with SQLite as the default database — fine for development, but not suitable for production. For the Doctor Finder project, switch to MySQL. You need one Python package (pymysql) and a change to the DATABASES dictionary in settings.py.

```
pip install pymysql
import pymysql  # top of settings.py

pymysql.install_as_MySQLdb()

DATABASES = {
    'default': {
```

Install the MySQL adapter for Python

Import pymysql before Django reads the database config

Makes pymysql behave like MySQLdb — required for Django compatibility

Replace the default SQLite block in settings.py

```
        'ENGINE': 'django.db.backends.mysql',   Tell Django to use MySQL
        'NAME': 'doctorfinder_db',              Your database name — create this in MySQL first
        'USER': 'root',                         MySQL username
        'PASSWORD': 'yourpassword',             MySQL password — never commit this to GitHub
        'HOST': 'localhost',                    Database host — localhost for local dev
        'PORT': '3306',                         Default MySQL port
    }
}
```

WARNING — Never Commit Passwords

Never put your real database password directly in settings.py if the file will be pushed to GitHub. Store it in a .env file, add .env to .gitignore, and load it using python-decouple: from decouple import config — then PASSWORD: config('DB_PASSWORD').

04 Running Migrations

Every time you create or change a model, Django must sync that change with the actual database. This two-step process — makemigrations then migrate — is Django's version control for your database schema. It tracks every change and can roll back if needed.

```
python manage.py makemigrations          Detects changes in models.py and creates a migration file (not the table yet)
python manage.py migrate                 Applies all pending migration files — this creates or alters the actual tables
python manage.py showmigrations          Lists every migration and whether it has been applied — [X] = applied
python manage.py migrate accounts 0001   Roll back to a specific migration if you need to undo a change
```

TRY THIS LIVE

Add name, specialization, city, and phone to your Doctor model. Run makemigrations, then migrate. Open MySQL Workbench and verify the table accounts_doctor was created with the correct columns. If you see a migration error, run showmigrations to find out what is and is not applied.

05 Basic ORM Queries

Once the table exists, you can create and retrieve records entirely in Python — no SQL required. Open the Django shell with python manage.py shell to experiment with ORM commands before using them in views.

```
from accounts.models import Doctor                    Import your model into the shell or view
Doctor.objects.all()                                  Retrieve every record — returns a QuerySet
Doctor.objects.create(name='Dr. Patel', specialization='Cardiology', city='Ahmedabad',
phone='9876543210')
    Insert a new record directly
Doctor.objects.filter(city='Ahmedabad')               Filter records — equivalent to WHERE city =
                                                      'Ahmedabad'
Doctor.objects.get(id=1)                              Fetch exactly one record by primary key — raises error
                                                      if not found
Doctor.objects.filter(city='Surat').count()           Count matching records without loading them all
```

CASE STUDY — Practo

Context — Practo is India's leading digital health platform with 20 million+ patients and 100,000+ doctors listed, including thousands of clinics in Ahmedabad, Surat, and Vadodara.

Challenge — They needed to store deeply interrelated medical data — doctors with multiple specializations, multi-city clinic locations, appointment time slots, and patient records — in a way that was fast to query and easy to evolve as the product grew.

What they did — Their data layer was built using Django models with properly chosen field types and relationships (ForeignKey, ManyToManyField). Filters like 'find cardiologists in Ahmedabad open today' became a single ORM query instead of hand-written SQL joins.

Result — The platform scaled city by city across India by adding data records, not restructuring schemas. New features like teleconsultation required model extensions, not rewrites.

Takeaway — A clean model design with the right field types at the start is the difference between 'add a column' and 'rewrite the database.'

Source: Practo Engineering — medium.com/practo-engineering (Django architecture publicly discussed)

WHERE TO GO NEXT

Your data layer is live — every future feature in Doctor Finder builds on top of what you defined today.

Explore deeper — Django Model field reference — docs.djangoproject.com/en/stable/ref/models/fields/

Try on your own — Add a Specialization model with just a name field, then add a ForeignKey from Doctor to Specialization. Run migrations and verify the relationship in MySQL Workbench.

Connect the dots — Session 05 uses Django Admin to give your models a full management UI — create, edit, search, and filter records without writing a single view.

Going further — Read about model Meta options (ordering, verbose_name, indexes) — these control how your model behaves in Admin, shell, and ORM queries at scale.

AI PROMPT — ASK CHATGPT / GEMINI / CLAUDE

Act as a Django data layer tutor. Explain what an ORM is in simple Hinglish using the analogy of a translator between Python and MySQL. Then show a 5-field Doctor model and one ORM filter query to find doctors in Ahmedabad. Keep it under 150 words.

SESSION
05

60 MIN · ADMIN & TOOLS

Django Admin Customization

Learning Objectives

- Enable Django's built-in admin panel and create a superuser account
- Register models so they appear in the admin interface
- Customise list_display to show relevant columns at a glance
- Add search_fields to allow quick record lookup by name or field
- Configure list_filter for one-click filtering by category or status
- Understand when to use admin.site.register versus the @admin.register decorator

01 What is Django Admin?

Django Admin is a fully working back-office management panel that Django generates for you automatically, based on your models. A clinic receptionist in Surat can add, edit, and delete doctor records through the admin interface without writing a single line of code. No separate admin panel to build — it is ready in under five minutes.

CONCEPT — Auto-Generated CRUD

Django reads your registered models and generates Create, Read, Update, and Delete screens automatically. Register a Doctor model today and you immediately get a working doctor management UI — list view, search bar, edit form, and delete confirmation. This is the same Admin system used by operations teams at Indian startups to manage data without developer involvement.

LINES OF CODE TO GET A WORKING ADMIN UI

1

admin.site.register(Doctor) — that is the entire setup

02 Enabling Admin & Creating a Superuser

Django Admin is already enabled by default. Check that django.contrib.admin is in INSTALLED_APPS in settings.py, and that the admin URL is included in urls.py. Then create your first admin account from the terminal using the createsuperuser command.

`python manage.py createsuperuser`	Prompts for username, email, and password — creates the first admin account
`python manage.py runserver`	Start the development server

```
# Visit: http://127.0.0.1:8000/admin/
```
Open this in your browser — log in with your superuser credentials

```
path('admin/', admin.site.urls),
```
This line must be in doctorfinder_project/urls.py — it is there by default

TRY THIS LIVE

Run createsuperuser. Start the server. Open /admin/ in your browser and log in. Explore the default interface — you will see Users and Groups already registered. Notice the list view, the search, and the add button. This is what we are about to customise for our own models.

03 Registering a Model

By default, your own models do not appear in admin. You must register them in admin.py inside your app. There are two ways to do this: a simple one-liner using admin.site.register(), or a more powerful decorator approach using @admin.register() which lets you attach customisation directly.

admin.site.register()

Quick one-liner. Good for getting started fast. Customisation is added separately by creating a ModelAdmin class and passing it as the second argument: admin.site.register(Doctor, DoctorAdmin).

@admin.register(Doctor)

Decorator style — cleaner and more Pythonic. The customisation class is defined directly below the decorator. Preferred in professional Django projects because the model and its admin config stay together.

```
from django.contrib import admin
```
Import admin — always at top of admin.py

```
from .models import Doctor
```
Import the model you want to manage

```
# Option 1 — Simple registration
admin.site.register(Doctor)
```
One line — gives a basic list with no customisation

```
# Option 2 — Decorator with customisation (preferred)
@admin.register(Doctor)
```
Attach this decorator to a ModelAdmin class below it

```
class DoctorAdmin(admin.ModelAdmin):
```
Define your customisations inside this class

```
    pass
```
Placeholder — replace with list_display, search_fields, etc.

04 Customising the List View

The default admin list shows only the string representation of each record (e.g. 'Doctor object (1)'). Four ModelAdmin attributes transform this into a genuinely useful management tool — add them inside your DoctorAdmin class and reload the admin page to see the difference immediately.

```
@admin.register(Doctor)
class DoctorAdmin(admin.ModelAdmin):
list_display = ['name', 'specialization', 'city', 'phone']
    Columns to show in the list view — left to right
search_fields = ['name', 'specialization']
    Adds a search bar — searches these fields with LIKE query
      list_filter = ['specialization', 'city']      Adds a right-side filter panel — click to filter by value
      ordering = ['name']                           Default sort order for the list — alphabetical by name
```

ModelAdmin Attribute	What It Does	Example Value
list_display	Columns shown in the list view	['name', 'specialization', 'city']
search_fields	Fields searched when using the search bar	['name', 'phone']
list_filter	Right-side panel for one-click filtering	['specialization', 'city']
ordering	Default sort order of the list	['name'] or ['-created_at']
list_per_page	Number of records per page (default 100)	25
readonly_fields	Fields shown but not editable in the form	['created_at']
date_hierarchy	Drill-down navigation by date	'created_at'

TRY THIS LIVE

Add the DoctorAdmin class with list_display = ['name', 'specialization', 'city', 'phone'], search_fields = ['name'], and list_filter = ['specialization']. Reload admin. Add two or three doctors via the + Add button. Then use the search bar and filter panel to verify they work.

05 Admin Best Practices

PRO TIP — Change the /admin/ URL in Production

The default /admin/ path is the first thing an attacker tries on any Django site. In production, rename it to something unpredictable: path('manage-doctorfinder/', admin.site.urls). This alone blocks automated brute-force login attempts from bots that scan for /admin/ across all Django deployments.

CASE STUDY — Instamojo

Context — Instamojo is a Bengaluru-based payment and e-commerce platform used by over a million small businesses across India, including sole proprietors and micro-sellers from Gujarat cities like Ahmedabad and Rajkot.

Challenge — Their operations and customer support team needed to manage merchants, verify accounts, track transactions, and process refunds — but involving a developer for every lookup was creating a bottleneck.

What they did — They invested in customising Django Admin with list_display for key merchant fields, search_fields on business name and phone number, list_filter by city and account status, and inline models to show linked transactions alongside each merchant record.

Result — The non-technical operations team could resolve merchant queries end-to-end without developer involvement, cutting resolution time significantly and freeing engineers to build features instead of answering data questions.

Takeaway — Thirty minutes invested in list_display and search_fields gives your operations team a full internal tool — admin is not just a debug panel.

Source: Instamojo Engineering Blog — tech.instamojo.com (Django usage and internal tooling)

"Django Admin is the first internal tool you will ever ship — make it useful, not just functional."

WHERE TO GO NEXT

You now have a working management UI for every model in Doctor Finder — no frontend code written.

Explore deeper — Django ModelAdmin full reference — docs.djangoproject.com/en/stable/ref/contrib/admin/

Try on your own — Add list_per_page = 10 and a date_hierarchy field to DoctorAdmin. Add ten doctors via admin and test paginating through them.

Connect the dots — Session 06 introduces Django Forms — you will build a public-facing 'Add Doctor' form that works alongside the admin panel.

Going further — Look into Django Admin actions — they let you apply bulk operations (approve, deactivate, export) to selected records in one click. Used heavily in content moderation and batch-processing workflows.

AI PROMPT — ASK CHATGPT / GEMINI / CLAUDE

Act as a Django admin tutor. Explain list_display, search_fields, and list_filter in simple Hinglish. Use a doctor management system as the example — show a complete DoctorAdmin class with all three attributes. Keep it under 150 words.

SESSION
06

60 MIN · USER INPUT

Django Forms Basics

Learning Objectives

- Explain why Django Form classes are safer than raw HTML forms
- Create a Form class with appropriate field types and validation
- Distinguish between GET and POST requests and use each correctly
- Handle form submission in a view using the POST-redirect-GET pattern
- Access validated data from form.cleaned_data in a view
- Display field-level and form-level validation errors in a template

01 Why Use Django Forms?

You could write a plain HTML <form> and read request.POST['name'] directly in the view. But Django's Form class gives you three things a plain form does not: automatic validation (is this a real email?), automatic sanitisation (strip harmful input), and ready-made error messages to show back to the user. Every production Django project — from Zomato's restaurant sign-up to Gujarat government portals — uses Form classes for any user input that touches a database.

CONCEPT — The Form Class

A Django Form is a Python class where each attribute is a form field. Django uses it to render HTML input elements, validate submitted data, and return cleaned, safe values. Define it once in forms.py and reuse it in any view or template.

Plain HTML Form

You write the HTML manually, read raw strings from request.POST, validate everything yourself, and risk missing edge cases — empty strings, SQL injection, wrong data types. One mistake and bad data reaches your database.

Django Form Class

Django renders the fields, validates types and constraints automatically, and returns cleaned_data only when everything is valid. Errors are collected per field and can be displayed directly in the template with one tag.

02 Creating a Form Class

Create a forms.py file inside your app. Each attribute you define becomes one input field in the final HTML form. Django maps field types to HTML inputs automatically — CharField becomes a text input, EmailField becomes an email input, BooleanField becomes a checkbox.

```
# accounts/forms.py                          Create this file inside your app folder
from django import forms                     Import Django's forms module
class AddDoctorForm(forms.Form):             Define the form as a Python class
    name = forms.CharField(max_length=100)   Renders as <input type='text'> — required by default
specialization = forms.CharField(max_length=100)
  Another text field
    city = forms.CharField(max_length=50)    Short text for city name
    phone = forms.CharField(max_length=15)   Phone as text — preserves leading zeros
    email = forms.EmailField()               Validates email format automatically before you even
                                             check
experience = forms.IntegerField(min_value=0, max_value=50)
  Integer with built-in range validation
```

Form Field	HTML Input Rendered	Built-in Validation
CharField()	text input	Required, max_length enforced
EmailField()	email input	Valid email format required
IntegerField()	number input	Must be a whole number; min_value / max_value optional
BooleanField()	checkbox	Must be checked (True) if required=True
ChoiceField(choices=...)	select dropdown	Value must be one of the given choices
DateField()	date input	Must parse to a valid date
CharField(widget=forms.Textarea)	textarea	Same as CharField but renders multi-line

03 GET vs POST — Which Goes Where

Every form works with two HTTP methods. GET is used to fetch and display the empty form. POST is used when the user submits data. A single Django view handles both by checking request.method — this is the standard pattern used across every Django project.

> **CONCEPT — GET vs POST**
>
> GET requests have no body — data goes in the URL (?search=patel). POST requests have a body — data is hidden from the URL and can be larger. Use GET for search and filters, POST for anything that creates, updates, or deletes data. Django's CSRF protection only applies to POST, PUT, PATCH, and DELETE.

WARNING — CSRF Token is Mandatory

Every HTML form that uses method='POST' must include {% csrf_token %} as the first line inside the <form> tag. If you omit it, Django will reject the submission with a 403 Forbidden error. This token prevents cross-site request forgery attacks — an attacker on another website cannot submit forms on behalf of your logged-in users.

04 Handling Submission in the View

The view does three jobs: show the empty form on GET, validate the submitted data on POST, and redirect on success. The redirect after a successful POST is important — it prevents the browser from resubmitting the form if the user refreshes the page. This is called the POST-Redirect-GET pattern.

```
# accounts/views.py
from django.shortcuts import render, redirect          render for templates, redirect for POST-Redirect-GET
from .forms import AddDoctorForm                       Import the form class from forms.py
def add_doctor(request):                               One view handles both GET (show form) and POST (submit)

    if request.method == 'POST':                       User clicked Submit
        form = AddDoctorForm(request.POST)             Bind submitted data to the form
        if form.is_valid():                            Django runs all field validators — True only if everything passes

            name = form.cleaned_data['name']           Access validated, sanitised values from cleaned_data
            city = form.cleaned_data['city']           cleaned_data only exists after is_valid() returns True
# Save to DB here (Session 07 will use form.save() instead)
            return redirect('doctor_list')             POST-Redirect-GET: redirect on success to prevent double submission

    else:                                              GET request — show an empty form
        form = AddDoctorForm()                         Create an unbound form (no data attached)
return render(request, 'add_doctor.html', {'form': form})
   Pass form to template — shown either empty or with errors
```

05 Rendering the Form & Displaying Errors

Django can render the entire form in one line ({{ form.as_p }}) or field by field for full layout control. Rendering field by field is preferred in real projects — it lets you wrap each input in Bootstrap classes and place error messages exactly where the designer intended.

```
<form method='POST'>                                   Always method='POST' for any form that changes data
  {% csrf_token %}                                     Security token — must be the first thing inside the form tag
```

```
  {{ form.as_p }}                                   Option 1: Render all fields wrapped in <p> tags — quick but less control

<!-- Option 2: Render field by field (recommended for styled forms) -->
  <div class='mb-3'>                                Bootstrap form group
    {{ form.name.label_tag }}                       Renders the <label> for the name field
    {{ form.name }}                                 Renders the <input> for the name field
    {{ form.name.errors }}                          Renders a <ul> of validation errors for this field, if any
  </div>
<button type='submit' class='btn btn-primary'>Add Doctor</button>
</form>
```

TRY THIS LIVE

Create AddDoctorForm with name, specialization, city, phone, and email fields. Build the add_doctor view with GET/POST handling. Create the template with {% csrf_token %} and {{ form.as_p }}. Submit the form with an invalid email address and confirm Django shows the error without crashing.

CASE STUDY — Razorpay Merchant Onboarding

Context — Razorpay is India's leading payment gateway, used by thousands of businesses across Gujarat — from Ahmedabad textile exporters to Surat diamond traders — to accept online payments.

Challenge — Their merchant sign-up collects sensitive business details: GST number, bank account, PAN, business category. Invalid data reaching their payment processing pipeline would cause failed payouts and compliance issues.

What they did — They built their onboarding using Django Form classes with custom validators — for example, a PAN field validator that checks the 10-character format (AAAAA9999A) before the data ever reaches their database or compliance checks.

Result — Invalid applications are caught at the form layer, not discovered later during KYC or payout processing. This reduced manual review overhead and improved the speed at which valid merchants get activated.

Takeaway — Validation at the form layer is the cheapest place to catch bad data — it costs one if statement, not a failed bank transaction.

Source: Razorpay Engineering Blog — engineering.razorpay.com (Django-based stack publicly documented)

WHERE TO GO NEXT

You can now safely collect, validate, and process user input — the backbone of any web application.

Explore deeper — Django Form fields and validators reference — docs.djangoproject.com/en/stable/ref/forms/fields/

Try on your own — Add a custom validator to the phone field that raises a ValidationError if the number is not exactly 10 digits. Test it by submitting a 5-digit number.

Connect the dots — Session 07 introduces ModelForm — a smarter form that is linked directly to your Doctor model and can save itself to the database in one line.

Going further — Read about Django formsets — they let you submit multiple form instances (e.g. add 5 doctors at once) in a single POST request.

AI PROMPT — ASK CHATGPT / GEMINI / CLAUDE

Act as a Django forms tutor. Explain the POST-Redirect-GET pattern in simple Hinglish. Use the example of adding a doctor's details to a hospital website. Explain why redirecting after POST prevents double submissions. Keep it under 150 words.

SESSION
07

60 MIN · USER INPUT

Django Model Forms

Learning Objectives

- Explain what a ModelForm is and how it differs from a regular Form
- Create a ModelForm linked to the Doctor model using class Meta
- Control which fields are included using fields or exclude
- Save a new record to the database using form.save()
- Pre-populate a form with existing data to support edit functionality
- Add Bootstrap styling to ModelForm fields using the widgets attribute

01 What is a ModelForm?

In Session 06 you defined every field in the form manually and then wrote code to save cleaned_data to the database. A ModelForm automates both steps. It reads your model definition and generates the form fields for you — and form.save() writes the validated data directly to the database in one line. It is the most common form pattern in production Django projects.

> **CONCEPT — ModelForm**
>
> A ModelForm is a Form that is permanently linked to a model via class Meta. It auto-generates fields that match the model's columns, inherits the model's max_length and blank rules as validators, and adds a save() method that handles the INSERT or UPDATE SQL for you. Think of it as the model and the form merged into one object.

Regular Form (Session 06)

Fields defined manually in forms.py. Validation rules set by hand. After is_valid(), you must read cleaned_data and call Doctor.objects.create() yourself. More code, more chances to forget a field.

ModelForm (Session 07)

Fields auto-generated from the model. Validation rules inherited from model field constraints. After is_valid(), one call to form.save() inserts or updates the database record. Less code, less duplication.

02 Creating DoctorForm as a ModelForm

Replace your existing forms.py AddDoctorForm with a ModelForm version. The inner class Meta is where you tell Django which model to link and which fields to expose. Using fields = '__all__' exposes every model column — in most projects you will list specific fields instead to avoid exposing internal fields like created_at or is_verified.

```
# accounts/forms.py
from django import forms
from .models import Doctor                          Import the model this form is linked to
class DoctorForm(forms.ModelForm):                  Inherit from ModelForm, not forms.Form
    class Meta:                                     Inner class that controls the model link and field selection
        model = Doctor                              Link this form to the Doctor model
fields = ['name', 'specialization', 'city', 'phone', 'email']
   Expose only these fields — internal fields like created_at stay hidden
# Alternative: exclude = ['created_at', 'is_verified']
   Expose everything EXCEPT these — use fields or exclude, not both
```

> **PRO TIP — Always Use fields, Not __all__**
>
> Avoid fields = '__all__' in production forms. If you add a sensitive column to your model later (is_admin, balance, verification_status), it will automatically appear in the public form and be submittable by anyone. Always list fields explicitly.

03 Using ModelForm in a View

The view structure is almost identical to Session 06 — GET shows the empty form, POST validates and saves. The only difference is that form.save() replaces the manual Doctor.objects.create() call. For editing an existing record, you pass the instance argument when creating the form.

◆ Add New Doctor — Create View

```
from django.shortcuts import render, redirect, get_object_or_404
from .forms import DoctorForm
from .models import Doctor
def add_doctor(request):                            Handles both GET (show form) and POST (save record)
    if request.method == 'POST':
        form = DoctorForm(request.POST)             Bind submitted data
        if form.is_valid():
            form.save()                             One line — validates and inserts the record into the database
            return redirect('doctor_list')          POST-Redirect-GET
    else:
        form = DoctorForm()                         Empty unbound form for GET
return render(request, 'add_doctor.html', {'form': form})
```

◆ Edit Existing Doctor — Update View

```
def edit_doctor(request, id):                    id comes from the URL — e.g. /doctors/edit/3/
    doctor = get_object_or_404(Doctor, id=id)    Fetch the record or return 404 if it does not exist
form = DoctorForm(request.POST or None, instance=doctor)
   instance pre-populates the form with existing data
    if form.is_valid():                          On POST, validates the submitted changes
        form.save()                              UPDATE query — saves changes to the existing record
        return redirect('doctor_list')
return render(request, 'edit_doctor.html', {'form': form})
   On GET, renders the pre-filled edit form
```

TRY THIS LIVE

Create DoctorForm as a ModelForm linked to your Doctor model. Build add_doctor and edit_doctor views. Add two URL patterns. Test add_doctor — confirm the record appears in Django Admin. Then test edit_doctor — change the city and verify the change persists after redirect.

04 Customising Fields with Widgets

By default, ModelForm renders plain HTML inputs with no CSS classes. To apply Bootstrap styling, add a widgets dictionary inside class Meta. A widget controls how a field renders in HTML — you can change input type, add CSS classes, set placeholder text, or switch a CharField to a dropdown.

```
class DoctorForm(forms.ModelForm):
    class Meta:
        model = Doctor
fields = ['name', 'specialization', 'city', 'phone']
        widgets = {                              Define how each field renders in HTML
            'name': forms.TextInput(attrs={
                'class': 'form-control',         Bootstrap class for styled text input
'placeholder': 'Enter doctor full name'
   Placeholder text inside the input
            }),
'specialization': forms.Select(attrs={'class': 'form-select'}),
   Renders as a dropdown if the field uses choices
'city': forms.TextInput(attrs={'class': 'form-control'}),
'phone': forms.TextInput(attrs={'class': 'form-control', 'maxlength': '10'}),
   HTML maxlength as extra frontend guard
        }
```

PRO TIP — Add Widgets in __init__ for Reusability

If many forms share the same Bootstrap class pattern, override __init__ and loop over self.fields to add 'form-control' to every field automatically. This way you never forget to style a new field and your Meta stays clean.

05 form.save() — Under the Hood

form.save() does different SQL depending on context. On a new form (no instance), it runs INSERT. On a form created with instance=doctor, it runs UPDATE on that specific record. Passing commit=False delays the database write and gives you a chance to set additional fields (like the logged-in user) before saving.

```
form.save()                                  INSERT (new record) or UPDATE (existing instance) — commits immediately

doctor = form.save(commit=False)             Creates the Doctor object in memory but does NOT save to DB yet
doctor.added_by = request.user               Set extra fields that are not in the form (e.g. the current user)
doctor.save()                                Now write to the database manually
```

CASE STUDY — Practo Doctor Onboarding

Context — Practo lists over 100,000 doctors across India, including hundreds of specialists in Ahmedabad, Surat, and Vadodara. Each doctor profile — with name, specialization, clinic address, fees, and availability — is created through an onboarding form.

Challenge — The form for onboarding a doctor maps almost exactly to their Doctor model — same fields, same constraints. Maintaining a separate Form class and a separate save() routine meant any model change required two matching updates, causing bugs when one was updated and the other was not.

What they did — They moved doctor onboarding to a ModelForm. Field definitions now live only in the model. Adding a new column (e.g. teleconsultation_available) automatically appears in the form with no extra code. commit=False was used to attach the clinic's account ID before saving.

Result — New doctor profile attributes could be shipped in a single model migration and a Meta fields list update — no separate form maintenance. Onboarding form bugs caused by model-form drift were eliminated.

Takeaway — When your form maps to a model, use a ModelForm — it removes an entire class of duplication bugs.

Source: Practo Engineering — medium.com/practo-engineering (Django ModelForm patterns publicly referenced)

“form.save() is one line. The code you did not write cannot have a bug.”

WHERE TO GO NEXT

You can now create and edit any database record through a form — the two most common user actions in any web application.

Explore deeper — Django ModelForm full reference — docs.djangoproject.com/en/stable/topics/forms/modelforms/

Try on your own — Add a profile_photo = models.ImageField to your Doctor model. Add it to DoctorForm fields. Update the view to handle request.FILES alongside request.POST. Upload a photo and verify it saves.

Connect the dots — Session 08 adds a full User Authentication system — registration and login are just ModelForms connected to Django's built-in User model.

Going further — Read about inline formsets (inlineformset_factory) — they let you add or edit multiple related records (e.g. a doctor's clinic locations) on a single form page.

AI PROMPT — ASK CHATGPT / GEMINI / CLAUDE

Act as a Django tutor. Compare a regular Form and a ModelForm side by side in simple Hinglish. Use the Doctor Finder project as the example. Show the key difference in how each saves data to the database. Keep it under 150 words.

SESSION
08

60 MIN · AUTHENTICATION

User Authentication System

Learning Objectives

- Explain what Django's built-in User model provides without any setup
- Build a registration view using UserCreationForm and auto-login after signup
- Authenticate and log in users using authenticate() and login()
- Log out users cleanly using Django's logout() function
- Protect views from unauthenticated access using the @login_required decorator
- Display the logged-in username in the navbar using request.user in templates

01 Django's Built-in User Model

Django ships with a fully built User model — you do not need to create one. It already has username, email, password (stored as a secure hash, never plain text), first_name, last_name, is_active, is_staff, and date_joined. Every Indian web application from Naukri to Meesho that runs on Django uses this same model as its starting point.

> **CONCEPT — Password Hashing**
>
> Django never stores a plain-text password. When a user registers, Django runs the password through PBKDF2 + SHA-256 and stores only the hash. When the user logs in, Django hashes what they typed and compares — the original password is never recoverable. You never write hashing code — Django does it automatically whenever you call user.set_password() or use the built-in forms.

User Field	Type	What It Stores
username	CharField	Unique login identifier — required
email	EmailField	Email address — not unique by default
password	CharField	PBKDF2 hash — never the real password
first_name / last_name	CharField	Display name — optional
is_active	BooleanField	False = account deactivated, cannot log in
is_staff	BooleanField	True = can access Django Admin
date_joined	DateTimeField	Set automatically on registration
last_login	DateTimeField	Updated automatically on each login

02 User Registration

Django provides UserCreationForm — a ready-made ModelForm linked to the User model. It includes username, password, and password confirmation fields with matching validation built in. You can use it directly or extend it to add email and other fields.

```
# accounts/forms.py — extend UserCreationForm to add email
from django.contrib.auth.forms import UserCreationForm
from django.contrib.auth.models import User
class RegisterForm(UserCreationForm):                    Extend the built-in form to add extra fields
    email = forms.EmailField(required=True)              Add email as a required field
    class Meta:
        model = User                                     Link to Django's built-in User model
fields = ['username', 'email', 'password1', 'password2']
   password1 = password, password2 = confirmation
```

```
# accounts/views.py — Registration view
from django.contrib.auth import login
from .forms import RegisterForm
def register(request):
    if request.method == 'POST':
        form = RegisterForm(request.POST)
        if form.is_valid():
            user = form.save()                           Creates and saves the User record — password is
                                                         hashed automatically
            login(request, user)                         Log the user in immediately after registration
            return redirect('home')
    else:
        form = RegisterForm()
return render(request, 'register.html', {'form': form})
```

> **TRY THIS LIVE**
>
> Create RegisterForm, build the register view, and add a URL. Register a test user. Open Django Admin -> Users and confirm the new account appears. Check the password column — it should show a hash starting with 'pbkdf2_sha256$', never your actual password.

03 Login & Logout

Django's authenticate() function checks a username and password against the database and returns the User object if they match (or None if they do not). After authenticating, call login() to start the session. For logout, call logout() — it

clears the session data and the user is no longer recognised by request.user.

```
# Login view
from django.contrib.auth import authenticate, login
from django.contrib.auth.forms import AuthenticationForm
    Django's built-in login form
def user_login(request):
    if request.method == 'POST':
form = AuthenticationForm(data=request.POST)
    Validates username + password against the DB
        if form.is_valid():
            user = form.get_user()              Returns the authenticated User object
            login(request, user)                Starts the session — request.user is now set
            return redirect('home')
    else:
        form = AuthenticationForm()
return render(request, 'login.html', {'form': form})
```

```
# Logout view
from django.contrib.auth import logout
def user_logout(request):
    logout(request)                     Clears session data — user is anonymous after this
    return redirect('login')            Send them to the login page
```

04 Protecting Views with @login_required

Any view that should only be accessible to logged-in users needs one decorator. If an unauthenticated user tries to access the URL, Django redirects them to the login page automatically. The login_url argument tells Django where your login page lives.

```
from django.contrib.auth.decorators import login_required
@login_required(login_url='/login/')            Unauthenticated users are redirected to /login/
                                                automatically
def dashboard(request):                         Only logged-in users reach this code
    return render(request, 'dashboard.html')
# In settings.py — set a global default redirect instead of repeating login_url
LOGIN_URL = '/login/'                           All @login_required decorators use this by default
```

> **PRO TIP — Set LOGIN_URL Once in settings.py**

Instead of passing login_url='/login/' to every @login_required decorator, set LOGIN_URL = '/login/' in settings.py once. Every decorator in the entire project automatically uses it. Change your login URL in one place, not fifty.

05 Showing User Info in Templates

Django's authentication middleware makes request.user available in every template automatically — you do not need to pass it through the view context. Use it to personalise the navbar, show/hide buttons, and greet users by name.

```
{% if user.is_authenticated %}                  True if someone is logged in
    <span>Hello, {{ user.username }}</span>     Show the logged-in username
    <a href='/logout/'>Logout</a>               Logout link — only shown to logged-in users
{% else %}
    <a href='/login/'>Login</a>                 Login link — only shown to guests
    <a href='/register/'>Register</a>
{% endif %}
```

TRY THIS LIVE

Add the login/logout views and URL patterns. Update base.html to show the username if authenticated and a Login link if not. Log in with your test user and confirm the navbar changes. Then visit a @login_required view without logging in and confirm you are redirected.

CASE STUDY — Meesho

Context — Meesho is an Indian social commerce platform with over 150 million users, used heavily by resellers in Gujarat cities like Rajkot, Ahmedabad, and Surat to sell products via WhatsApp and Instagram.

Challenge — They needed a secure registration and login system that could handle millions of accounts, protect seller data, and prevent unauthorised access to order history and earnings dashboards.

What they did — Their authentication layer is built on Django's User model and auth system. Password hashing (PBKDF2), session management, and login-required protection are all provided by Django out of the box — the team focused on product features rather than rebuilding security primitives.

Result — Millions of seller accounts are protected with industry-standard password hashing and session security without a single line of custom cryptographic code.

Takeaway — Django's auth system is production-grade from day one — do not reinvent it, just configure it.

Source: Meesho Engineering Blog — medium.com/meesho-engineering (Django stack publicly referenced)

WHERE TO GO NEXT

Your Doctor Finder project now has working registration, login, logout, and route protection.

Explore deeper — Django authentication full reference — docs.djangoproject.com/en/stable/topics/auth/

Try on your own — Add a profile page view decorated with @login_required that shows the logged-in user's username and email. Link to it from the navbar.

Connect the dots — Session 09 uses this auth system to trigger a welcome email automatically when a new user registers.

Going further — Read about Django's AbstractUser — it lets you extend the built-in User model with custom fields (phone number, profile photo, role) without losing any of the built-in auth functionality.

AI PROMPT — ASK CHATGPT / GEMINI / CLAUDE

Act as a Django auth tutor. Explain in simple Hinglish how Django stores passwords as hashes and why this matters. Use the analogy of a one-way lock. Then show the three lines of code that register, log in, and log out a user. Keep it under 150 words.

SESSION
09

60 MIN · COMMUNICATION

Email Sending in Django

Learning Objectives

- Configure Django to send email through an SMTP server
- Send a plain text email using send_mail()
- Send an HTML email using EmailMultiAlternatives
- Build an HTML email body from a Django template using render_to_string()
- Trigger a welcome email automatically when a new user registers
- Store credentials securely and handle email errors in production

01 How Django Email Works

Django does not send email itself — it connects to an SMTP server that does the actual delivery. In development, you configure Gmail's SMTP. In production, you use a transactional email service like SendGrid or Mailgun. Django's email functions are the same either way — only the settings change. Every Indian SaaS product sends transactional email this way: OTPs, booking confirmations, invoices.

CONCEPT — SMTP and Transactional Email

SMTP (Simple Mail Transfer Protocol) is the standard protocol for sending email over the internet. Transactional email means emails triggered by user actions — registration, password reset, booking confirmation — as opposed to marketing campaigns. Django's mail module wraps SMTP so you can send transactional email in one function call.

Development Setup

Use Gmail SMTP or Django's console backend (EMAIL_BACKEND = 'django.core.mail.backends.console.EmailBackend'). Console backend prints email to the terminal instead of sending it — perfect for testing without a real SMTP server.

Production Setup

Use a transactional email service — SendGrid, Mailgun, or Amazon SES. These services give you reliable delivery, open/click tracking, and bounce handling. Gmail SMTP has a 500 email/day limit and is not suitable for production.

02 SMTP Configuration in settings.py

All email settings live in settings.py. For Gmail SMTP in development, you need to enable 2-Factor Authentication on your Google account and generate an App Password — a 16-character code that replaces your real password for SMTP connections.

```
# settings.py — Gmail SMTP for development
EMAIL_BACKEND = 'django.core.mail.backends.smtp.EmailBackend'
    Use real SMTP — set to 'console.EmailBackend' to print instead of send
EMAIL_HOST = 'smtp.gmail.com'                    Gmail's outgoing mail server
EMAIL_PORT = 587                                 Port for TLS (recommended over 465/SSL for Gmail)
EMAIL_USE_TLS = True                             Encrypt the connection — always True with port 587
EMAIL_HOST_USER = config('EMAIL_HOST_USER')      Your Gmail address — loaded from .env via
                                                 python-decouple
EMAIL_HOST_PASSWORD = config('EMAIL_HOST_PASSWORD')
    Gmail App Password — NOT your real Gmail password
DEFAULT_FROM_EMAIL = 'DoctorFinder <noreply@doctorfinder.com>'
    The From: name and address shown to recipients
```

WARNING — Use App Password, Not Your Gmail Password

Gmail blocks SMTP login with your real password. Go to myaccount.google.com -> Security -> 2-Step Verification -> App Passwords. Generate one for 'Mail' and use that 16-character code as EMAIL_HOST_PASSWORD. Store it in .env and never commit it to GitHub.

03 Sending a Plain Text Email

For simple notifications — OTP messages, quick alerts, system messages — send_mail() is the fastest option. It takes a subject, message body, sender, and a list of recipients. Call it from any view or utility function.

```
from django.core.mail import send_mail
send_mail(
    subject='Welcome to Doctor Finder',          Email subject line
message='Hello! Your account has been created successfully.',
    Plain text body
    from_email='noreply@doctorfinder.com',       Sender address (overrides DEFAULT_FROM_EMAIL if
                                                 set)
    recipient_list=['user@example.com'],         Always a list — even for a single recipient
    fail_silently=False,                         True = swallow errors silently; False = raise exception
                                                 on failure
)
```

TRY THIS LIVE

Set EMAIL_BACKEND to the console backend first. Add a send_mail() call inside your register view after form.save(). Register a test user and check your terminal — the full email content should print there. Once it looks right, switch to real SMTP and send to your own email address.

04 HTML Email with Templates

Plain text emails are functional but not professional. For welcome emails and booking confirmations, you want a styled HTML email — a logo, a button, a greeting with the user's name. Django's EmailMultiAlternatives lets you attach both a plain text fallback and an HTML version. Email clients automatically pick whichever they support.

```
# Create templates/emails/welcome.html — your HTML email layout
from django.core.mail import EmailMultiAlternatives
from django.template.loader import render_to_string
    Renders a .html template to a string
def send_welcome_email(user):                    Call this function from the register view after saving the user
subject = f'Welcome to Doctor Finder, {user.first_name or user.username}!'
text_body = f'Hi {user.username}, your account is ready.'
    Plain text fallback for email clients that block HTML
html_body = render_to_string('emails/welcome.html', {'user': user})
    Render the HTML template with the user object as context
email = EmailMultiAlternatives(subject, text_body, 'noreply@doctorfinder.com',
[user.email])
email.attach_alternative(html_body, 'text/html')
    Attach the HTML version — clients prefer this over plain text
    email.send()                                 Send the email
```

> **PRO TIP — Keep HTML Emails Simple**
>
> HTML emails are not like web pages — most email clients block external CSS and <style> tags. Write all CSS inline (style='color: #333;'). Use a single-column layout with max-width 600px. Avoid JavaScript entirely — it is always blocked. Test your email in both Gmail and a phone client before going live.

05 Triggering Email on Registration

Connecting email to registration is a one-line addition to the register view — call send_welcome_email(user) right after form.save(). In production, you would move this to a background task (Celery) so the user is not waiting for the SMTP server to respond before being redirected. For the Doctor Finder project, a direct call is sufficient.

```
# Updated register view — session_08 view with email added
def register(request):
    if request.method == 'POST':
        form = RegisterForm(request.POST)
        if form.is_valid():
            user = form.save()                   Save user to database
```

```
            send_welcome_email(user)          Send welcome email — add after save, before login
            login(request, user)              Log in automatically
            return redirect('home')
    else:
        form = RegisterForm()
return render(request, 'register.html', {'form': form})
```

CASE STUDY — Urban Company

Context — Urban Company (formerly UrbanClap) is India's largest home services platform, with service professionals across Ahmedabad, Surat, and Vadodara offering salon, cleaning, plumbing, and appliance repair bookings.

Challenge — Every booking triggers multiple email notifications — confirmation to the customer, job alert to the professional, cancellation notices, and review requests. These needed to look professional, include booking details dynamically, and arrive reliably.

What they did — They built HTML email templates using Django's render_to_string, with booking data passed as context. EmailMultiAlternatives ensured customers on older email clients always received a readable plain-text fallback. For reliability, email sending was moved to a Celery background queue.

Result — Customers receive consistently branded, dynamic confirmation emails within seconds of booking. The same email template system handles all notification types — one infrastructure, multiple use cases.

Takeaway — render_to_string turns your email body into a template problem — the same Django skills that build web pages build professional email content.

Source: Urban Company Engineering Blog — medium.com/urban-company-engineering

WHERE TO GO NEXT

Your application can now communicate with users automatically — every new registration gets a welcome email.

Explore deeper — Django email documentation — docs.djangoproject.com/en/stable/topics/email/

Try on your own — Build a templates/emails/welcome.html with inline CSS. Add the user's name, a 'Find a Doctor' button, and the DoctorFinder logo. Send it to your own email and test how it looks on your phone.

Connect the dots — Session 10 uses email to deliver password reset OTPs — the same send_mail() function you used today.

Going further — Read about Django Signals (post_save on User) — they let you trigger send_welcome_email without touching the register view, keeping concerns separated.

AI PROMPT — ASK CHATGPT / GEMINI / CLAUDE

Act as a Django email tutor. Explain in simple Hinglish the difference between send_mail() and EmailMultiAlternatives. Use the example of a welcome email for a doctor booking app. Show the five key

settings.py lines needed for Gmail SMTP. Keep it under 150 words.

SESSION
10

60 MIN · AUTHENTICATION

Forgot Password & Session Management

Learning Objectives

- Explain what Django sessions are and how they persist data across requests
- Store, retrieve, and delete values in the session using request.session
- Generate a 6-digit OTP and store it securely in the session
- Build a three-step forgot password flow: request OTP -> verify OTP -> reset password
- Configure session expiry using SESSION_COOKIE_AGE
- Explain what middleware is and how SessionMiddleware and AuthenticationMiddleware work

01 What are Django Sessions?

HTTP is stateless — every request is independent and the server remembers nothing between them. Django sessions solve this by assigning each browser a unique session ID stored in a cookie, and linking that ID to a data dictionary stored on the server. This is how Django remembers who is logged in, what is in a cart, and what OTP was sent — across multiple page loads.

> **CONCEPT — Session vs Cookie**
>
> A cookie is a small value stored in the user's browser. Django stores only a session ID in the cookie — a random key. The actual data (OTP, cart items, user preferences) lives in Django's session store on the server. Even if a user sees their cookie, they only see the ID — not the data it points to.

Session Operation	Code	What It Does
Store a value	request.session['otp'] = '482931'	Saves OTP to session — survives across page loads
Retrieve a value	request.session.get('otp')	Returns value or None — safe even if key does not exist
Delete one key	del request.session['otp']	Remove a key once it has been used
Clear entire session	request.session.flush()	Deletes all session data and the session ID — used on logout
Check if key exists	'otp' in request.session	Returns True / False
Set expiry	request.session.set_expiry(300)	This session expires in 300 seconds — 0 means browser close

PRO TIP — Always Use .get() to Read Session Values

Use request.session.get('otp') instead of request.session['otp']. If the key does not exist (expired session, first visit, cleared data), the square-bracket version raises a KeyError and crashes. .get() returns None safely — your view can handle that gracefully.

02 Session Configuration in settings.py

Sessions are enabled by default in every Django project. The two settings you will adjust most often are where session data is stored and how long a session lasts. The default storage is the database (django_session table created by migrate). The default lifetime is two weeks.

```
# settings.py
SESSION_ENGINE = 'django.contrib.sessions.backends.db'
```
Default — stores session data in the database (django_session table)

```
SESSION_COOKIE_AGE = 60 * 60 * 24 * 7
```
Session lifetime in seconds — this is 7 days (default is 2 weeks)

```
SESSION_EXPIRE_AT_BROWSER_CLOSE = True
```
If True, session ends when the browser is closed regardless of AGE

```
SESSION_COOKIE_SECURE = True
```
Production only: cookie sent only over HTTPS — prevents interception

```
SESSION_COOKIE_HTTPONLY = True
```
Default True: blocks JavaScript from reading the session cookie

03 Building the Forgot Password Flow

A password reset flow has three pages: Step 1 — enter email and receive OTP, Step 2 — enter the OTP, Step 3 — set the new password. The OTP and the email being reset are stored in the session to carry state across these three views without exposing them in the URL.

◆ Step 1 — Request OTP

```
import random
from django.contrib.auth.models import User
from django.core.mail import send_mail
def forgot_password(request):
    if request.method == 'POST':
        email = request.POST.get('email')
otp = str(random.randint(100000, 999999))
```
Generate a 6-digit OTP

```
        request.session['reset_otp'] = otp          Store OTP in session — not in the URL, not in the DB
request.session['reset_email'] = email
    Store the email being reset
        request.session.set_expiry(600)             OTP expires in 10 minutes
send_mail('Your OTP', f'Your password reset OTP is: {otp}',
'noreply@doctorfinder.com', [email])
        return redirect('verify_otp')               Redirect regardless — do not confirm whether email
                                                    exists
return render(request, 'forgot_password.html')
```

WARNING — Do Not Confirm Email Existence

Always redirect to the OTP page regardless of whether the email is registered. If you show 'Email not found', an attacker can enumerate which emails have accounts on your platform. Show the same message for both found and not-found emails — 'If that email is registered, an OTP has been sent.'

◆ Step 2 — Verify OTP

```
from django.contrib import messages
def verify_otp(request):
    if request.method == 'POST':
        entered_otp = request.POST.get('otp')
stored_otp = request.session.get('reset_otp')
    Retrieve OTP from session — returns None if expired
        if not stored_otp:
messages.error(request, 'OTP expired. Please request a new one.')
    Session expired — send them back
return redirect('forgot_password')
        if entered_otp == stored_otp:
request.session['otp_verified'] = True
    Mark OTP as verified — gates entry to the reset page
            return redirect('reset_password')
messages.error(request, 'Invalid OTP. Please try again.')
    return render(request, 'verify_otp.html')
```

◆ Step 3 — Reset Password

```
def reset_password(request):
if not request.session.get('otp_verified'):
    Guard: only reachable after a verified OTP
        return redirect('forgot_password')          Block direct URL access
```

```
    if request.method == 'POST':
new_password = request.POST.get('password')
email = request.session.get('reset_email')
        user = User.objects.get(email=email)
        user.set_password(new_password)          Hashes the new password automatically — never set
                                                 user.password directly
        user.save()
        del request.session['reset_otp']         Clean up — OTP can never be reused
        del request.session['reset_email']
        del request.session['otp_verified']
messages.success(request, 'Password reset successful. Please log in.')
        return redirect('login')
return render(request, 'reset_password.html')
```

TRY THIS LIVE

Build all three views and URL patterns. Use the console email backend so OTPs print to the terminal. Register a user with an email, then go through the full forgot password flow: request OTP -> copy OTP from terminal -> verify -> set new password -> log in with the new password. Confirm it works end to end.

04 Middleware — How Sessions and Auth Work

Middleware is code that runs automatically on every request and every response — before your view and after it. Django's MIDDLEWARE list in settings.py shows which layers are active. Two middleware classes make sessions and authentication work throughout your entire project without you doing anything per view.

```
# settings.py — these two must be in MIDDLEWARE for auth and sessions to work
'django.contrib.sessions.middleware.SessionMiddleware',
    Parses the session cookie on every request and makes request.session available
'django.contrib.auth.middleware.AuthenticationMiddleware',
    Reads the session and attaches the correct User object to request.user on every request
# Order matters — SessionMiddleware must come before AuthenticationMiddleware
    Auth needs the session to be loaded first
```

CONCEPT — Middleware as a Pipeline

Think of middleware as a series of checkpoints every request must pass through on the way in, and every response passes through on the way out. SessionMiddleware reads the cookie on the way in. AuthenticationMiddleware uses the session to attach request.user. By the time your view runs, both are already set up.

CASE STUDY — HDFC NetBanking OTP Flow

Context — HDFC Bank's internet banking platform is used by millions of customers across Gujarat and India for fund transfers, bill payments, and account management — all requiring secure authentication.

Challenge — Password reset over the internet must be secure without requiring a physical branch visit. The challenge is verifying that the person resetting the password is the real account holder — without storing the OTP permanently in the database where it could be exposed in a breach.

What they did — OTPs for password reset are generated server-side, stored temporarily in the server's session store (not the main database), sent to the registered mobile number, and expired after a short window (typically 5-10 minutes). The session is cleared immediately after the OTP is used.

Result — Customers reset passwords securely online in under 2 minutes. OTPs never persist in the main database — a breach of customer records would not expose any active OTPs.

Takeaway — Session storage for OTPs is the right pattern precisely because it is temporary and server-side — the user never sees it, and it self-destructs on expiry.

Source: HDFC Bank Security Practices — hdfcbank.com/security; OTP session pattern is standard in Indian banking applications

---

“Store the OTP in the session, not the URL. What the user cannot see, they cannot tamper with.”

WHERE TO GO NEXT

Doctor Finder now has a complete authentication system — register, login, logout, and self-service password reset.

Explore deeper — Django sessions documentation — docs.djangoproject.com/en/stable/topics/http/sessions/

Try on your own — Add a resend OTP link to the verify_otp page. If the user clicks it, generate a new OTP, update the session value, and send a new email — without the user having to re-enter their email.

Connect the dots — Session 11 covers media file uploads and user profiles — the next layer of a real user account system.

Going further — Read about Django's built-in password reset views (PasswordResetView, PasswordResetConfirmView) — they implement a token-based reset via email link, which is more secure than OTP for some use cases.

AI PROMPT — ASK CHATGPT / GEMINI / CLAUDE

Act as a Django security tutor. Explain in simple Hinglish why we store the OTP in the session instead of passing it in the URL. Use the analogy of a locker vs a notepad. Then show the three lines to store, verify, and delete an OTP from the session. Keep it under 150 words.

SESSION
11

60 MIN · FILE HANDLING

Media Files & User Profile

Learning Objectives

- Distinguish between static files (CSS/JS) and media files (user uploads)
- Configure MEDIA_ROOT and MEDIA_URL in settings.py
- Add an ImageField to a model and organise uploads into subfolders
- Handle file uploads in a view using request.FILES
- Serve uploaded media files during development
- Build a profile edit page that updates photo, name, and phone

01 Static Files vs Media Files

Static files are assets you create as a developer — CSS, JavaScript, icons. They never change at runtime. Media files are assets uploaded by users at runtime — profile photos, doctor certificates, clinic images. Django handles both, but through separate configurations. Mixing them up is one of the most common beginner mistakes.

Static Files

Created by the developer. Stored in the static/ folder. Served via STATIC_URL. Collected with collectstatic for production. Examples: style.css, logo.png, app.js.

Media Files

Uploaded by users at runtime. Stored in the media/ folder. Served via MEDIA_URL. Must be backed up independently. Examples: profile photos, uploaded documents, clinic images.

02 Configuring MEDIA_ROOT and MEDIA_URL

Two settings control where uploaded files are saved and how they are accessed in a URL. MEDIA_ROOT is the absolute folder path on the server's filesystem. MEDIA_URL is the URL prefix that maps to that folder. Together they form a complete upload address: a file saved to media/profiles/photo.jpg is accessible at /media/profiles/photo.jpg.

```
# settings.py
MEDIA_URL = '/media/'                    URL prefix for all uploaded files
MEDIA_ROOT = BASE_DIR / 'media'          Absolute folder path where Django saves uploaded files

# doctorfinder_project/urls.py — add this at the bottom
from django.conf import settings
from django.conf.urls.static import static
urlpatterns += static(settings.MEDIA_URL, document_root=settings.MEDIA_ROOT)
    Serves media files in development — remove in production (use Nginx/S3)
```

WARNING — Development Only

The static() URL pattern in urls.py only works when DEBUG = True. In production, Django does not serve media files — a web server like Nginx or a cloud storage service like AWS S3 handles this. Never use Django's dev media serving in production.

03 Adding ImageField to a Model

ImageField stores the file path in the database (not the file itself) and saves the actual file to MEDIA_ROOT. The upload_to argument creates a subfolder inside media/ — use it to keep uploads organised. Django requires Pillow to be installed to use ImageField.

```
pip install Pillow                                  Required for ImageField — without this, Django raises an error

# accounts/models.py — add a UserProfile model
class UserProfile(models.Model):
user = models.OneToOneField(User, on_delete=models.CASCADE)
   One profile per user — links to Django's built-in User
phone = models.CharField(max_length=15, blank=True)
    profile_photo = models.ImageField(
        upload_to='profiles/',                      Files saved to media/profiles/filename.jpg
        blank=True,                                 Photo is optional — do not require it on registration
        null=True                                   Allows NULL in the database when no photo is uploaded

    )
    def __str__(self):
return f'{self.user.username} Profile'
```

04 Handling File Uploads in a View

File uploads require two changes compared to regular form handling: the HTML form needs enctype='multipart/form-data', and the view must pass request.FILES alongside request.POST when binding the form. Missing either one means the uploaded file is silently ignored.

```
# accounts/forms.py
class ProfileForm(forms.ModelForm):
    class Meta:
        model = UserProfile
        fields = ['phone', 'profile_photo']
```

```
# accounts/views.py
@login_required
def edit_profile(request):
profile, created = UserProfile.objects.get_or_create(user=request.user)
   Create profile if it doesn't exist yet
    if request.method == 'POST':
form = ProfileForm(request.POST, request.FILES, instance=profile)
   request.FILES carries the uploaded photo — do not forget this
        if form.is_valid():
            form.save()
            return redirect('profile')
    else:
        form = ProfileForm(instance=profile)
return render(request, 'profile.html', {'form': form, 'profile': profile})
```

```
<!-- profile.html — form must have enctype for file uploads to work -->
<form method='POST' enctype='multipart/form-data'>
   enctype tells the browser to send files as binary data
  {% csrf_token %}
  {{ form.as_p }}
  <button type='submit'>Save Profile</button>
</form>
<!-- Display uploaded photo if it exists -->
{% if profile.profile_photo %}
<img src='{{ profile.profile_photo.url }}' width='100'>
   .url generates the /media/profiles/photo.jpg URL automatically
{% endif %}
```

TRY THIS LIVE

Create UserProfile, run migrations, build the edit_profile view. Upload a photo from the form. Check that it appears in your media/profiles/ folder on disk. Then display it in the template using profile.profile_photo.url. If the image does not load, verify that the media URL pattern is added to urls.py.

CASE STUDY — Naukri.com Resume & Photo Uploads

Context — Naukri.com is India's largest job portal with over 70 million registered job seekers, including hundreds of thousands of candidates from Gujarat cities applying for IT, manufacturing, and finance roles.

Challenge — Every candidate can upload a resume (PDF), a profile photo, and supporting documents. These files needed to be stored reliably, served quickly, and kept separate from the application's static assets to avoid conflicts during deployments.

What they did — User-uploaded files are stored in a dedicated media storage layer (separate from static

assets), with upload_to paths organised by user ID and file type — /media/resumes/{user_id}/ and /media/photos/{user_id}/. In production, files are served from a CDN rather than the application server.

Result — Profile photos and resumes load fast for recruiters, file storage scales independently of the application, and a deployment or code rollback never touches candidate files.

Takeaway — Keep user-uploaded files logically and physically separate from your application code — MEDIA_ROOT exists for exactly this reason.

Source: Naukri Engineering Blog — naukri.com/blog/engineering (file handling architecture publicly discussed)

WHERE TO GO NEXT

Doctor Finder now supports user profiles with photos — a core feature of any health platform.

Explore deeper — Django ImageField and FileField reference — docs.djangoproject.com/en/stable/ref/models/fields/#imagefield

Try on your own — Add a default profile photo that shows when no photo has been uploaded — use an {% if profile.profile_photo %} check in the template and display a placeholder image from your static folder otherwise.

Connect the dots — Session 12 builds full CRUD for the Doctor model — create, list, edit, and delete records through views the public can access.

Going further — Read about storing media files on AWS S3 using django-storages — the standard approach for production Django applications that need scalable, CDN-backed file storage.

AI PROMPT — ASK CHATGPT / GEMINI / CLAUDE

Act as a Django file upload tutor. Explain in simple Hinglish why a form needs enctype='multipart/form-data' for file uploads, and why the view needs request.FILES. Use the analogy of sending a parcel vs sending a letter. Keep it under 150 words.

SESSION
12

60 MIN · CORE WEB PATTERNS

CRUD Operations

Learning Objectives

- Implement all four CRUD views — Create, Read (list + detail), Update, Delete — for the Doctor model
- Build a doctor listing page with links to individual doctor detail pages
- Use ModelForm for both create and update views with a single shared template
- Build a delete confirmation page that uses a POST request to confirm deletion
- Wire up clean, RESTful-style URL patterns for all CRUD routes
- Apply @login_required to protect write operations from unauthenticated users

01 What is CRUD?

CRUD stands for Create, Read, Update, Delete — the four operations that every data-driven web application needs. A hospital management system in Ahmedabad, a product catalogue for a Surat textile exporter, and a student tracker at TOPS all perform these same four operations on their data. Mastering CRUD with Django means you can build the core of any business application.

Operation	HTTP Method	Django View Does	URL Pattern Example
Create	GET + POST	Show form, then save new record	/doctors/add/
Read — List	GET	Fetch all records, render list template	/doctors/
Read — Detail	GET	Fetch one record by ID, render detail template	/doctors/3/
Update	GET + POST	Show pre-filled form, then save changes	/doctors/3/edit/
Delete	GET + POST	Show confirmation page, then delete on POST	/doctors/3/delete/

02 Read — List and Detail Views

```
# accounts/views.py
from django.shortcuts import render, get_object_or_404, redirect
from .models import Doctor
```

```
from .forms import DoctorForm

def doctor_list(request):                          Show all doctors
doctors = Doctor.objects.all().order_by('name')
   Fetch every record, sorted alphabetically
return render(request, 'doctors/list.html', {'doctors': doctors})

def doctor_detail(request, id):                    Show one doctor's full profile
    doctor = get_object_or_404(Doctor, id=id)      Returns 404 automatically if the ID does not exist
return render(request, 'doctors/detail.html', {'doctor': doctor})
```

03 Create and Update — Shared Form View

Create and Update share the same form and almost the same view logic. The only difference is whether an instance is passed to the form. You can write two separate views (clean and explicit) or one combined view that checks for an ID in the URL. Two separate views is the recommended approach for beginners — easier to read and debug.

```
@login_required
def doctor_create(request):                        Add a new doctor
    form = DoctorForm(request.POST or None)        request.POST or None: bound on POST, unbound on
                                                   GET
    if form.is_valid():
        form.save()
        return redirect('doctor_list')
return render(request, 'doctors/form.html', {'form': form, 'title': 'Add Doctor'})

@login_required
def doctor_update(request, id):                    Edit an existing doctor
    doctor = get_object_or_404(Doctor, id=id)
form = DoctorForm(request.POST or None, instance=doctor)
   instance pre-fills the form with existing data
    if form.is_valid():
        form.save()
return redirect('doctor_detail', id=doctor.id)
return render(request, 'doctors/form.html', {'form': form, 'title': 'Edit Doctor'})
```

> **PRO TIP — One Form Template for Create and Update**
>
> Pass a title variable ('Add Doctor' or 'Edit Doctor') as context and use it in the template's heading and submit button. Both views point to the same doctors/form.html — one template, zero duplication. When the form design changes, you update one file.

04 Delete — Confirmation Page Pattern

Never delete a record on a GET request. If someone shares or bookmarks /doctors/3/delete/, a browser prefetch could accidentally delete the record. Always show a confirmation page on GET, and perform the deletion only on a POST. This is the standard Django delete pattern.

```
@login_required
def doctor_delete(request, id):
    doctor = get_object_or_404(Doctor, id=id)
    if request.method == 'POST':                    Delete only on confirmed POST — never on GET
        doctor.delete()                             Permanently removes the record from the database
        return redirect('doctor_list')
return render(request, 'doctors/confirm_delete.html', {'doctor': doctor})
   GET: show confirmation page
```

```
<!-- doctors/confirm_delete.html -->
<p>Are you sure you want to delete <strong>{{ doctor.name }}</strong>?</p>
<form method='POST'>                                POST triggers the actual deletion
  {% csrf_token %}
<button type='submit' class='btn btn-danger'>Yes, Delete</button>
   Confirm button
<a href='{% url "doctor_list" %}' class='btn btn-secondary'>Cancel</a>
   Cancel goes back to the list — no deletion
</form>
```

05 URL Patterns for All CRUD Routes

```
# accounts/urls.py
from django.urls import path
from . import views
urlpatterns = [
path('doctors/', views.doctor_list, name='doctor_list'),
   List all doctors
path('doctors/<int:id>/', views.doctor_detail, name='doctor_detail'),
   Single doctor detail page
path('doctors/add/', views.doctor_create, name='doctor_create'),
   Add new doctor
path('doctors/<int:id>/edit/', views.doctor_update, name='doctor_update'),
   Edit doctor
```

```
path('doctors/<int:id>/delete/', views.doctor_delete, name='doctor_delete'),
    Delete confirmation + deletion
]
```

TRY THIS LIVE

Wire up all five URL patterns and views. Add 3 doctors using the Create form. Edit one to change the city. Delete another through the confirmation page. Confirm the list updates correctly after each operation. Check Django Admin to verify the database matches what the views show.

CASE STUDY — Zoho (Zoho CRM)

Context — Zoho is a Chennai-based SaaS company and one of India's largest software exporters. Their Zoho CRM product is used by thousands of Gujarat businesses to manage sales leads, contacts, and follow-ups.

Challenge — CRM is fundamentally a CRUD application — salespeople create leads, read their pipeline, update deal stages, and delete duplicate records. The interface had to be fast, reliable, and forgiving (delete confirmation before permanent removal).

What they did — Every data entity in Zoho CRM follows the same CRUD pattern: a list view with search and filter, a detail view, an inline edit view using form pre-population, and a soft-delete with confirmation. The delete confirmation step alone has prevented thousands of accidental data loss incidents.

Result — The predictable CRUD pattern means any Zoho CRM user can navigate any module — Leads, Contacts, Deals — without relearning the interface, because every section works the same way.

Takeaway — Consistency in CRUD patterns is a UX feature — once a user knows how Add/Edit/Delete works in one section, they know how it works everywhere.

Source: Zoho Engineering Blog — blog.zoho.com/engineering (CRUD and product architecture publicly discussed)

WHERE TO GO NEXT

Doctor Finder now has a complete, working data management interface for any user with an account.

Explore deeper — Django's get_object_or_404 and shortcuts — docs.djangoproject.com/en/stable/topics/http/shortcuts/

Try on your own — Add a search bar to the doctor list that filters results by name. Pass request.GET.get('q') to the view and use Doctor.objects.filter(name__icontains=q) when a query is present.

Connect the dots — Session 13 upgrades the delete operation to use AJAX — no page reload, no separate confirmation page, instant feedback.

Going further — Read about Django's class-based views (ListView, DetailView, CreateView, UpdateView, DeleteView) — they implement the same CRUD patterns in fewer lines of code and are used heavily in production projects.

AI PROMPT — ASK CHATGPT / GEMINI / CLAUDE

Act as a Django CRUD tutor. Explain in simple Hinglish why delete should never happen on a GET request. Use the analogy of a shopkeeper accidentally deleting an order because a WhatsApp link was previewed. Then show the two-step delete view pattern in Django. Keep it under 150 words.

SESSION
13

60 MIN · DYNAMIC UI
CRUD Using AJAX

Learning Objectives

- Explain what AJAX is and why it improves user experience over full page reloads
- Return JSON responses from Django views using JsonResponse
- Send a POST request from JavaScript using the Fetch API
- Pass the CSRF token correctly in AJAX requests
- Implement AJAX-based delete that removes a card from the DOM without reloading
- Show dynamic success and error messages after AJAX operations

01 What is AJAX and Why Use It?

AJAX (Asynchronous JavaScript and XML) lets a webpage send requests to the server and update part of the page — without reloading the whole thing. When a Swiggy user removes an item from their cart, the page does not reload — the item disappears instantly. That is AJAX. In Doctor Finder, AJAX-based delete removes a doctor card instantly without the user leaving the list page.

CONCEPT — JSON as the Language of AJAX

In a regular request, Django returns an HTML page. In an AJAX request, Django returns a JSON object — just data, no markup. JavaScript reads the JSON, decides what to do, and updates the DOM itself. Django's JsonResponse makes this a single line: return JsonResponse({'status': 'success'}).

Traditional Delete (Session 12)

Click Delete -> load confirmation page -> click confirm -> Django deletes -> redirect to list page. Three full page loads. Works on every device and browser, even without JavaScript.

AJAX Delete (Session 13)

Click Delete -> JavaScript sends POST in the background -> Django deletes -> JavaScript removes the card from the page. Zero page reloads. Instant feedback. Requires JavaScript to be enabled.

02 Returning JSON from a Django View

A Django view that serves AJAX requests returns JsonResponse instead of render(). JsonResponse accepts a Python dictionary and serialises it to JSON automatically. Always include a status key in your response so JavaScript can check whether the operation succeeded or failed.

```
# accounts/views.py
from django.http import JsonResponse
```

Django's built-in JSON response class

```
from django.views.decorators.http import require_POST
   Reject non-POST requests with 405 automatically

@login_required
@require_POST                                   This view accepts POST only — no confirmation page
                                                needed
def doctor_delete_ajax(request, id):
    doctor = get_object_or_404(Doctor, id=id)
    doctor_name = doctor.name                   Save name before deleting — can't access it after
    doctor.delete()
    return JsonResponse({
        'status': 'success',                    JavaScript checks this key to confirm deletion
'message': f'{doctor_name} has been removed.'
   Human-readable confirmation message
    })
```

03 The CSRF Token in AJAX Requests

Every Django POST request must include a CSRF token. In HTML forms, {% csrf_token %} handles this. In AJAX requests, there is no form — you must read the token from the cookie and send it manually in the request header. Without it, Django returns a 403 Forbidden error.

```
// JavaScript — read CSRF token from cookie
function getCsrfToken() {
    const name = 'csrftoken';
const cookies = document.cookie.split(';');
    for (let cookie of cookies) {
if (cookie.trim().startsWith(name + '=')) {
return cookie.trim().split('=')[1];
        }
    }
    return null;
}
```

04 Sending the AJAX Delete Request

The Fetch API is the modern JavaScript way to send HTTP requests — it is built into every browser and needs no external library. Call fetch() with the URL, set method to POST, pass the CSRF token in the headers, and handle the response in a .then() chain.

```
// JavaScript — AJAX delete with Fetch API
function deleteDoctor(doctorId, cardElement) {
    cardElement = the HTML card to remove from the DOM
if (!confirm('Remove this doctor?')) return;
    Quick browser confirm — lightweight alternative to a full confirmation page
fetch(`/doctors/${doctorId}/delete-ajax/`, {
        method: 'POST',                          Must be POST — Django rejects GET for
                                                 @require_POST views
        headers: {
            'X-CSRFToken': getCsrfToken(),       CSRF token in the header — Django reads this
                                                 automatically
'Content-Type': 'application/json',
        },
    })
    .then(response => response.json())           Parse the JSON response from Django
    .then(data => {
        if (data.status === 'success') {
            cardElement.remove();                Remove the card from the DOM — no page reload
            showToast(data.message);             Show a success notification to the user
        }
    })
.catch(error => showToast('Something went wrong.'));
    Handle network errors gracefully
}
```

05 Wiring It Into the Template

```
<!-- doctors/list.html — each card has an ID and a delete button -->
{% for doctor in doctors %}
<div class='card' id='doctor-card-{{ doctor.id }}'>
    ID used by JavaScript to find and remove the card
    <h5>{{ doctor.name }}</h5>
<p>{{ doctor.specialization }} — {{ doctor.city }}</p>
<button onclick="deleteDoctor({{ doctor.id }}, this.closest('.card'))">
    Pass ID and card reference to the JS function
        Delete
    </button>
</div>
{% endfor %}
<!-- Toast notification container -->
<div id='toast' style='display:none;'></div>     Shown temporarily after AJAX operations
```

TRY THIS LIVE

Add the doctor_delete_ajax view and URL. Open your browser's Developer Tools -> Network tab. Click a Delete button and watch the POST request and JSON response appear in the Network tab — you will see Django's response without any page reload. Confirm the card disappears from the list.

PRO TIP — Always Keep a Traditional Fallback

Build the traditional confirmation page delete (Session 12) first, then add AJAX as an enhancement. If a user has JavaScript disabled — common on older mobile networks in rural Gujarat — they can still delete records using the HTML form. AJAX should enhance, not replace.

CASE STUDY — Swiggy Cart Management

Context — Swiggy is India's leading food delivery platform, processing millions of orders daily from cities across Gujarat including Ahmedabad, Surat, Vadodara, and Rajkot.

Challenge — When a customer removes an item from their cart or changes quantity, a full page reload would reset scroll position, re-fetch the entire restaurant menu, and feel sluggish — especially on mobile connections.

What they did — All cart operations — add, remove, update quantity — are handled with AJAX. The server returns a JSON response with the updated cart total and item count. JavaScript updates only the cart icon and price summary in the header, leaving the rest of the page untouched.

Result — Cart interactions feel instant — users on 4G connections in Tier-2 cities experience the same responsiveness as desktop users on broadband. Swiggy's conversion rates benefit directly from the reduced friction.

Takeaway — AJAX is not just a performance trick — for mobile-first Indian users on variable connections, it is the difference between a frustrating and a usable experience.

Source: Swiggy Engineering Blog — bytes.swiggy.com (frontend AJAX patterns publicly documented)

WHERE TO GO NEXT

Your Doctor Finder list page now deletes records instantly — a noticeably more professional user experience.

Explore deeper — MDN Fetch API reference — developer.mozilla.org/en-US/docs/Web/API/Fetch_API

Try on your own — Add an AJAX-based status toggle — a button that marks a doctor as available or unavailable and updates the card colour instantly without reloading. Return the new status in the JSON response.

Connect the dots — Session 14 dives into ORM QuerySets — you will use filter(), Q objects, and pagination to power the doctor search and listing features.

Going further — Read about Django REST Framework (covered in Module 5) — it provides a structured way to build JSON APIs, replacing hand-written JsonResponse views with serializers, viewsets, and routers.

AI PROMPT — ASK CHATGPT / GEMINI / CLAUDE

Act as a Django AJAX tutor. Explain in simple Hinglish what happens step by step when a user clicks an AJAX delete button — from the browser, to Django, back to JavaScript, to the DOM. Use the analogy of ordering on Swiggy without the page refreshing. Keep it under 150 words.

SESSION
14

60 MIN · DATA LAYER

Django ORM — QuerySet Deep Dive

Learning Objectives

- Chain filter(), exclude(), and order_by() to build precise queries
- Use Q() objects to write OR conditions and complex multi-field filters
- Use annotate() to add computed fields to each record in a QuerySet
- Use aggregate() to compute summary values across an entire QuerySet
- Use select_related() and prefetch_related() to eliminate N+1 query problems
- Implement pagination using Django's Paginator to break long lists into pages

01 QuerySets Are Lazy

A QuerySet does not hit the database when you write it — it is evaluated only when you iterate over it, convert it to a list, or render it in a template. This laziness lets you chain as many filters as you need before any SQL is executed. The final SQL sent to MySQL is a single optimised query, not one query per filter call.

CONCEPT — Lazy Evaluation

Doctor.objects.filter(city='Ahmedabad').filter(specialization='Cardiology').order_by('name') builds up a query description in memory. Django sends one SQL query to the database only when the template loops over the results. Chain freely — it costs nothing until evaluation.

```
# Basic chaining
Doctor.objects.all()                                    Every record — evaluates to SELECT * FROM
                                                        accounts_doctor
Doctor.objects.filter(city='Ahmedabad')                 WHERE city = 'Ahmedabad'
Doctor.objects.filter(city='Ahmedabad', specialization='Cardiology')
    AND condition — two filters in one call
Doctor.objects.exclude(city='Surat')                    WHERE NOT city = 'Surat' — opposite of filter
Doctor.objects.filter(name__icontains='patel')
    Case-insensitive LIKE — icontains is one of Django's field lookups
Doctor.objects.order_by('name')                         ORDER BY name ASC
Doctor.objects.order_by('-experience')                  ORDER BY experience DESC — minus sign reverses
                                                        direction
Doctor.objects.filter(city='Ahmedabad').order_by('name')[:10]
    LIMIT 10 — slicing a QuerySet maps to SQL LIMIT
```

Lookup	SQL Equivalent	Example

Lookup	SQL Equivalent	Example
exact (default)	= 'value'	filter(city='Ahmedabad')
iexact	ILIKE 'value'	filter(city__iexact='ahmedabad')
contains	LIKE '%value%'	filter(name__contains='Patel')
icontains	ILIKE '%value%'	filter(name__icontains='patel')
startswith	LIKE 'value%'	filter(name__startswith='Dr')
gt / gte	> / >=	filter(experience__gte=5)
lt / lte	< / <=	filter(fee__lt=500)
in	IN (...)	filter(city__in=['Ahmedabad', 'Surat'])
isnull	IS NULL	filter(profile_photo__isnull=True)

02 Q Objects — OR Conditions

Multiple arguments inside filter() are combined with AND. To combine conditions with OR — or to build NOT conditions — you need Q objects. Import Q from django.db.models and combine Q objects with the | (OR) and & (AND) operators and ~ (NOT).

```
from django.db.models import Q
# Find doctors in Ahmedabad OR Surat
Doctor.objects.filter(Q(city='Ahmedabad') | Q(city='Surat'))
   | means OR
# Find cardiologists in Ahmedabad OR any doctor in Surat
Doctor.objects.filter(Q(city='Ahmedabad', specialization='Cardiology') |
Q(city='Surat'))
   Mix of AND (comma) and OR (pipe)
# Search bar — match name OR specialization OR city
q = request.GET.get('q', '')
results = Doctor.objects.filter(
Q(name__icontains=q) | Q(specialization__icontains=q) | Q(city__icontains=q)
   One search query, three fields — exactly like Practo's doctor search
)
```

TRY THIS LIVE

Add a search bar to your doctor list. In the view, read request.GET.get('q'). If q exists, filter with three Q

conditions (name, specialization, city). If not, return Doctor.objects.all(). Pass the results to the template and verify the search finds matches in any of the three fields.

03 annotate() and aggregate()

aggregate() computes a single summary value across the entire QuerySet — like the average consultation fee across all doctors. annotate() adds a computed value to each individual record — like the number of appointments each doctor has. Both use Django's built-in database functions: Count, Avg, Sum, Max, Min.

```
from django.db.models import Count, Avg, Max, Min, Sum
# aggregate() – one value for the whole QuerySet
Doctor.objects.aggregate(avg_fee=Avg('fee'))      Returns {'avg_fee': 450.0} — average fee across all doctors
Doctor.objects.aggregate(max_exp=Max('experience'), min_exp=Min('experience'))
    Multiple aggregations in one query
# annotate() – adds a value to each record
Doctor.objects.annotate(num_appointments=Count('appointment'))
    Each Doctor object now has .num_appointments attribute
# Use the annotated field in ordering and filtering
Doctor.objects.annotate(num_appt=Count('appointment')).order_by('-num_appt')
    Doctors ranked by number of appointments — most booked first
```

04 select_related & prefetch_related

When you loop over doctors and access doctor.specialization.name for each one, Django runs a separate SQL query per doctor — this is the N+1 problem. With 100 doctors, that is 101 database queries. select_related() and prefetch_related() solve this by fetching related data in one or two queries instead.

```
# N+1 problem – DO NOT do this
doctors = Doctor.objects.all()  # 1 query
# In the template: {{ doctor.specialization.name }}  # +1 query per doctor = N+1

# select_related – for ForeignKey and OneToOne (SQL JOIN)
doctors = Doctor.objects.select_related('specialization').all()
    Fetches doctors + specializations in ONE query using JOIN

# prefetch_related – for ManyToMany and reverse FK (separate query)
doctors = Doctor.objects.prefetch_related('appointments').all()
    Fetches all related appointments in a SECOND query — still far better than N+1
```

PRO TIP — Use Django Debug Toolbar

Install django-debug-toolbar in development. It shows every SQL query your page executed, how long each took, and highlights duplicate queries. This is how you discover N+1 problems before they reach production. pip install django-debug-toolbar, then follow the docs to add it to INSTALLED_APPS and MIDDLEWARE.

05 Pagination with Django Paginator

Loading 500 doctors on a single page is slow and unusable. Django's Paginator splits a QuerySet into pages of a fixed size. Pass the page number from the URL query string (?page=2) and Paginator returns only the records for that page.

```
from django.core.paginator import Paginator
def doctor_list(request):
    q = request.GET.get('q', '')
doctors = Doctor.objects.filter(name__icontains=q) if q else Doctor.objects.all()
    doctors = doctors.order_by('name')
    paginator = Paginator(doctors, 10)          Show 10 doctors per page
    page_number = request.GET.get('page')       Read ?page= from the URL
page_obj = paginator.get_page(page_number)
   get_page handles invalid/missing page numbers gracefully
return render(request, 'doctors/list.html', {'page_obj': page_obj, 'q': q})
```

```
<!-- Pagination controls in template -->
{% if page_obj.has_previous %}
<a href='?page={{ page_obj.previous_page_number }}&q={{ q }}'>Previous</a>
   Preserve search query across page changes
{% endif %}
<span>Page {{ page_obj.number }} of {{ page_obj.paginator.num_pages }}</span>
{% if page_obj.has_next %}
<a href='?page={{ page_obj.next_page_number }}&q={{ q }}'>Next</a>
{% endif %}
```

CASE STUDY — Practo Doctor Search

Context — Practo's doctor search is used by patients across India to find specialists by location, availability, fee range, and rating — a multi-filter search across 100,000+ doctor profiles.

Challenge — A single search query needs to filter by city, one or more specializations, fee range, availability, and rating — simultaneously. Without ORM optimisation, each additional filter added another slow sequential query.

What they did — They built the search using Q objects for OR conditions (multiple cities or specializations), field lookups for range filters (fee__lte, rating__gte), select_related() for joining specialization data, and Paginator to serve results 20 per page. annotate() adds computed ranking scores per doctor.

Result — Practo's search returns filtered, ranked, paginated results in under 200ms even across a large dataset — because the entire query is compiled into a single optimised SQL statement before hitting the database.

Takeaway — Every Practo search filter is a Django Q object or field lookup chained together — the ORM does the optimisation, you write the Python.

Source: Practo Engineering — medium.com/practo-engineering (ORM and search architecture discussed)

WHERE TO GO NEXT

Doctor Finder now has a production-quality search, filter, and pagination system.

Explore deeper — Django QuerySet API full reference — docs.djangoproject.com/en/stable/ref/models/querysets/

Try on your own — Add fee range filters to the search — two number inputs (min fee, max fee) that use filter(fee__gte=min, fee__lte=max). Combine with the existing Q-based name search.

Connect the dots — Session 15 adds role-based access — some features will only be available to logged-in doctors, others only to patients or admins.

Going further — Read about Django's values() and values_list() — they return dictionaries or tuples instead of model instances, which is faster when you only need a subset of fields from a large QuerySet.

AI PROMPT — ASK CHATGPT / GEMINI / CLAUDE

Act as a Django ORM tutor. Explain the N+1 query problem in simple Hinglish using the analogy of a shopkeeper calling a supplier separately for every item on a list. Then show how select_related() fixes it in one line. Keep it under 150 words.

SESSION
15

60 MIN · AUTHENTICATION

Django Authentication — Advanced

Learning Objectives

- Implement role-based access by checking user group membership in views and templates
- Create Groups and assign users to them programmatically
- Use @permission_required to restrict views to users with specific permissions
- Build role-based dashboards that show different content to doctors, patients, and admins
- Write a custom decorator that enforces role-based access cleanly
- Restrict the Django Admin panel to staff users only and understand is_staff vs is_superuser

01 Beyond Login — Who Are They?

@login_required only checks whether someone is logged in. A real application needs to know what they are allowed to do once logged in. A patient should see their appointments but not add doctors. A doctor should manage their own profile but not access other doctors' records. An admin should see everything. This is role-based access control.

CONCEPT — Groups and Permissions

Django's auth system has two layers: Groups (named collections of users — Doctor, Patient, Admin) and Permissions (specific actions — can add doctor, can view patient record). You assign a user to a Group, and the Group carries a set of Permissions. Views and templates check either group membership or specific permissions.

Groups

Named roles: Doctor, Patient, Staff. A user belongs to one or more groups. Checked with request.user.groups.filter(name='Doctor').exists(). Good for role-based dashboard routing — show different pages to different roles.

Permissions

Granular actions: accounts.add_doctor, accounts.delete_appointment. Auto-generated by Django for every model (add, change, delete, view). Checked with request.user.has_perm('accounts.add_doctor'). Good for fine-grained feature access.

02 Creating Groups and Assigning Users

Groups can be created in Django Admin (easiest), in a data migration (recommended for production), or programmatically in the shell or a signal. Once a group exists, assign users to it at registration time based on which type of account they are creating.

```
from django.contrib.auth.models import Group, Permission
# Create groups (run once — in a migration or management command)
doctor_group, _ = Group.objects.get_or_create(name='Doctor')
```

get_or_create is safe to run multiple times

```
patient_group, _ = Group.objects.get_or_create(name='Patient')

# Assign a user to a group at registration
def register_doctor(request):
    if request.method == 'POST':
        form = RegisterForm(request.POST)
        if form.is_valid():
            user = form.save()
doctor_group = Group.objects.get(name='Doctor')
            user.groups.add(doctor_group)
            login(request, user)
return redirect('doctor_dashboard')
```

Add user to Doctor group immediately after registration

03 Checking Roles in Views and Templates

Once users are in groups, views and templates can check group membership to show the right content or block access entirely. Write a helper function to keep group checks clean and reusable across multiple views.

```
# accounts/utils.py — reusable role check helpers
def is_doctor(user):
return user.groups.filter(name='Doctor').exists()
def is_patient(user):
return user.groups.filter(name='Patient').exists()

# Protect a view — allow only doctors
from django.contrib.auth.decorators import user_passes_test
@login_required
@user_passes_test(is_doctor, login_url='/unauthorized/')
```

Redirects to /unauthorized/ if is_doctor returns False

```
def doctor_dashboard(request):
return render(request, 'dashboards/doctor.html')
```

```
<!-- Template — show different content by role -->
{% if request.user.groups.all|dictsort:'name'|first == 'Doctor' %}
<!-- OR use a custom template tag / context variable -->
{% if user_is_doctor %}
```

Pass user_is_doctor=is_doctor(request.user) from the view context

```
    <a href='/add-doctor/'>Add Doctor</a>        Only doctors see this link
{% elif user_is_patient %}
<a href='/my-appointments/'>My Appointments</a>
   Only patients see this
{% endif %}
```

04 Using @permission_required

Django auto-creates four permissions for every model: add, change, delete, and view. Custom permissions can be added in model Meta. @permission_required checks whether a user has a specific permission — more granular than group checks, useful when different roles share some but not all actions.

```
# Model Meta custom permissions
class Appointment(models.Model):
    class Meta:
        permissions = [
('can_approve_appointment', 'Can approve appointments'),
   Custom permission — format: (codename, description)
        ]

# View using @permission_required
from django.contrib.auth.decorators import permission_required
@permission_required('accounts.can_approve_appointment', raise_exception=True)
   raise_exception=True returns 403 instead of redirecting to login
def approve_appointment(request, id):
# Only users with this permission can reach this view
    ...

# Check permissions in a view manually
if request.user.has_perm('accounts.add_doctor'):
   Programmatic permission check — useful inside view logic
    # Show add doctor button
```

05 Role-Based Dashboards and Admin Access

The cleanest architecture for role-based dashboards is a single /dashboard/ URL that checks the user's role and redirects to the appropriate page. This way your navbar only needs one dashboard link regardless of who is logged in.

```
@login_required
def dashboard_router(request):                    # One URL — routes to the right dashboard based on role

    user = request.user
    if user.is_superuser:
        return redirect('/admin/')                # Superuser goes straight to Django Admin
elif user.groups.filter(name='Doctor').exists():
        return redirect('doctor_dashboard')
elif user.groups.filter(name='Patient').exists():
        return redirect('patient_dashboard')
    else:
        return redirect('home')                   # Fallback for users with no assigned role
```

User Type	is_staff	is_superuser	Groups	Can Access Admin?
Regular patient	False	False	Patient	No
Doctor	False	False	Doctor	No
Staff operator	True	False	Staff (optional)	Yes — limited to registered models
Superuser	True	True	Any	Yes — full admin access, bypass all permission checks

WARNING — is_staff vs is_superuser

is_staff only grants access to Django Admin. It does not mean the user has all permissions — they only see models explicitly granted to them. is_superuser bypasses all permission checks and sees everything. Never set is_superuser for regular staff accounts — give them is_staff and assign specific model permissions instead.

TRY THIS LIVE

Create Doctor and Patient groups in Django Admin. Register two test users — assign one to Doctor, one to Patient. Build the dashboard_router view. Confirm each user is redirected to the right dashboard. Then try accessing /add-doctor/ as the patient user — verify they are blocked.

CASE STUDY — Practo's Role-Based Platform

Context — Practo serves three distinct user types on one platform: patients who search and book, doctors who manage their profiles and schedules, and clinic admins who manage staff and billing — all using the

same Django application.

Challenge — A patient must never see another patient's records. A doctor must manage their own profile but not edit another doctor's. Clinic admins need billing data but not clinical records. A single login system needs to serve all three with completely different interfaces.

What they did — Users are assigned to named groups (Patient, Doctor, ClinicAdmin) at registration. Views use @user_passes_test decorators with role-check functions. Templates receive a role context variable and use it to conditionally render navigation, buttons, and data. Django Admin is restricted to internal Practo staff with is_staff.

Result — Three completely different user experiences are served from one Django codebase. Access control bugs — the kind that let patients see other patients' data — are eliminated at the decorator level rather than inside view logic.

Takeaway — Role-based access built on Django Groups keeps your permission logic in one place and out of your templates and business logic.

Source: Practo Engineering — medium.com/practo-engineering (multi-role architecture publicly discussed)

"The right information to the right person — role-based access is not a feature, it is a trust system."

WHERE TO GO NEXT

Doctor Finder now has a complete, secure, role-aware authentication system — the same architecture used in production health platforms.

Explore deeper — Django Groups and Permissions reference — docs.djangoproject.com/en/stable/topics/auth/default/#groups

Try on your own — Add a custom permission can_book_appointment to the Appointment model. Assign it to the Patient group. Protect the book_appointment view with @permission_required. Test that only patients can book.

Connect the dots — Session 16 adds third-party integrations — Google Login, email APIs, and SMS — building on the auth system you now have fully in place.

Going further — Read about Django's AbstractBaseUser and BaseUserManager — they let you replace the default User model entirely (using email as the login field instead of username), recommended when starting a new project that needs non-standard auth.

AI PROMPT — ASK CHATGPT / GEMINI / CLAUDE

Act as a Django auth tutor. Explain the difference between Groups and Permissions in simple Hinglish. Use the analogy of a hospital where doctors, receptionists, and patients all have different ID cards that open different doors. Show how to check group membership in a Django view in two lines. Keep it under 150 words.

SESSION
16

60 MIN · THIRD-PARTY INTEGRATIONS

Django Integrations — Social Login, Email & SMS

Learning Objectives

- Set up Google OAuth2 social login in Django using django-allauth
- Send transactional emails from Django views using Mailgun SMTP
- Send SMS OTPs to Indian mobile numbers using the Twilio API
- Store all API credentials securely using environment variables
- Restrict views to authenticated users regardless of login method

01 Why Every Real App Needs Integrations

No production web app builds everything from scratch. Practo, India's leading doctor-discovery platform, lets patients sign in with Google and verifies their mobile numbers with OTP before every appointment — two integrations you will build today in Doctor Finder.

SOCIAL LOGIN PREFERENCE

73%

Source: Auth0 State of Identity Report 2023 — share of users who prefer social login over creating a new password

02 Google Social Login — django-allauth

CONCEPT — OAuth2 in Four Steps

1) User clicks Sign in with Google. 2) Django redirects them to Google's consent screen. 3) After approval, Google sends an auth code to your callback URL. 4) django-allauth exchanges that code for a token, creates or fetches the Django user, and logs them in. You never see or store any passwords.

```
pip install django-allauth                                    Install allauth
INSTALLED_APPS += ['django.contrib.sites', 'allauth', 'allauth.account',
'allauth.socialaccount', 'allauth.socialaccount.providers.google']
   settings.py — add all five apps
SITE_ID = 1                                                   Required by allauth's Sites framework
AUTHENTICATION_BACKENDS = ['django.contrib.auth.backends.ModelBackend',
'allauth.account.auth_backends.AuthenticationBackend']
   settings.py — register both backends
```

```
LOGIN_REDIRECT_URL = '/dashboard/'                Where to redirect after successful login
path('accounts/', include('allauth.urls'))        urls.py — mount all allauth routes
```

TRY THIS LIVE

Go to console.cloud.google.com -> New Project -> Enable Google People API -> Credentials -> Create OAuth 2.0 Client ID (Web Application) -> set Authorised Redirect URI to http://127.0.0.1:8000/accounts/google/login/callback/ -> copy Client ID and Secret -> add them in Django Admin under Sites > Social Applications.

03 Transactional Email — Mailgun SMTP

Mailgun's free tier gives 5,000 emails per month — enough for welcome messages and appointment confirmations in any student project. It plugs directly into Django's email settings with no new SDK required.

```
pip install python-decouple                       For reading .env variables safely
EMAIL_BACKEND = 'django.core.mail.backends.smtp.EmailBackend'
    settings.py — use SMTP backend
EMAIL_HOST = 'smtp.mailgun.org'                   Mailgun SMTP server
EMAIL_PORT = 587                                  TLS port
EMAIL_USE_TLS = True                              Encrypt the connection
EMAIL_HOST_USER = config('MAILGUN_USER')          Read from .env — never hardcode
EMAIL_HOST_PASSWORD = config('MAILGUN_PASS')      Read from .env
from django.core.mail import send_mail            views.py — import
send_mail('Appointment Confirmed', 'Dr. Shah will see you at 3 PM.',
'noreply@clinic.com', [patient.email])
    Call this after a successful booking
```

PRO TIP — Protect Your API Keys

Create a .env file in your project root and add it to .gitignore before your first commit. Store MAILGUN_USER, MAILGUN_PASS, TWILIO_SID, and TWILIO_TOKEN there. A leaked key on a public GitHub repo can generate bills of thousands of rupees from cloud providers — this has happened to real students and professionals.

04 SMS OTP Verification — Twilio

CONCEPT — OTP Verification Flow

1) Patient submits their mobile number. 2) Django generates a 6-digit code, stores it in the session with a 10-minute expiry, and calls Twilio. 3) Twilio delivers the SMS. 4) Patient enters the code. 5) Django compares -> if valid, grants access. This is exactly how Ola, Swiggy, and BookMyShow verify Indian phone numbers.

```
pip install twilio                                  Install Twilio SDK
from twilio.rest import Client                      views.py — import
import random, datetime                             For OTP generation and expiry
otp = random.randint(100000, 999999)                Generate 6-digit OTP
request.session['otp'] = str(otp)                   Store in session
request.session['otp_expiry'] = str(datetime.datetime.now() +
datetime.timedelta(minutes=10))
    Set 10-minute expiry
client = Client(config('TWILIO_SID'), config('TWILIO_TOKEN'))
    Initialise Twilio client
client.messages.create(body=f'Your OTP for Doctor Finder is {otp}. Valid 10 min.',
from_=config('TWILIO_FROM'), to=f'+91{mobile}')
    Send SMS — prepend +91 for all Indian numbers
```

CASE STUDY — Practo Patient Onboarding

Context — Practo is India's largest digital health platform, serving patients in Ahmedabad, Surat, Vadodara, and 100+ Indian cities.

Challenge — Practo needed to reduce registration drop-off — patients abandoning the signup form — while also verifying Indian mobile numbers to enable SMS appointment reminders.

What they did — Practo added Google OAuth2 for one-tap login and Twilio-powered SMS OTP for mobile verification, exactly the two integrations built in this session.

Result — Registration completion improved because patients could sign up in one Google tap instead of filling a long form, and verified phone numbers reduced appointment no-shows.

Takeaway — Social login plus SMS OTP is the standard Indian consumer-app stack. You have just built it.

Source: Practo Engineering Blog, practo.com/engineering

Integration	Tool / Package	Free Tier	Use in Doctor Finder
Social Login	django-allauth + Google OAuth2	Free — no Google charge	One-tap patient signup
Transactional Email	Django + Mailgun SMTP	5,000 emails/month	Appointment confirmations, welcome email
SMS OTP	Twilio Python SDK	~1,000 SMS on trial credit	Mobile number verification

Integration	Tool / Package	Free Tier	Use in Doctor Finder
WhatsApp OTP	Twilio / Gupshup	Limited sandbox	Higher open rates — common in Indian health apps

WHERE TO GO NEXT

Doctor Finder now authenticates patients three ways: email/password, Google, and SMS OTP.

Explore deeper — django-allauth full docs at allauth.readthedocs.io, especially the headless mode for future React/Flutter frontends

Try on your own — Add a Resend OTP button that rate-limits to one SMS per 60 seconds — store the last send timestamp in the session

Connect the dots — Next session wires in Razorpay so patients can pay at the time of booking

Going further — Explore Firebase Authentication as an alternative that unifies social login, OTP, and token management in one SDK

AI PROMPT — ASK CHATGPT / GEMINI / CLAUDE

Act as a Django backend developer. Explain OAuth2 social login in simple Hinglish — from the moment a user clicks Sign in with Google to the moment Django creates their session. Mention what django-allauth handles automatically. Use the example of a doctor booking app. Keep it under 150 words.

SESSION
17

60 MIN · PAYMENT INTEGRATION

Payment Gateway Integration — Razorpay

Learning Objectives

- Understand the three-step payment flow: order creation, payment capture, and verification
- Integrate Razorpay into a Django project using its Python SDK
- Build a Book Appointment -> Pay Now -> Confirmation user journey
- Verify Razorpay payment signatures server-side to prevent fraud
- Handle both payment success and payment failure in Django views

01 How Online Payments Work

When a patient pays Rs 500 on a doctor booking app, three systems talk to each other in under two seconds: your Django server, Razorpay's servers, and the patient's bank. Understanding this flow is essential before writing a single line of payment code.

CONCEPT — The Three-Step Payment Flow

Step 1 — Order: Your Django server calls Razorpay to create an order with the amount. Razorpay returns an order_id. Step 2 — Payment: Razorpay's JavaScript checkout opens in the browser. The patient pays via UPI, card, or net banking. Razorpay returns a payment_id and signature to your page. Step 3 — Verify: Your server verifies the HMAC-SHA256 signature using your secret key. Only then mark the appointment as confirmed. Never skip Step 3.

RAZORPAY ANNUAL VOLUME

Rs 10L Cr+

Source: Razorpay Annual Report 2023 — total payments processed across India per year

02 Setting Up Razorpay

```
pip install razorpay                                    Install Razorpay Python SDK
RAZORPAY_KEY_ID = config('RAZORPAY_KEY_ID')             settings.py — read from .env
RAZORPAY_KEY_SECRET = config('RAZORPAY_KEY_SECRET')
    settings.py — read from .env
import razorpay                                         views.py — import SDK
```

```
rz_client = razorpay.Client(auth=(settings.RAZORPAY_KEY_ID,
settings.RAZORPAY_KEY_SECRET))
```
Initialise client — reuse across views

TRY THIS LIVE

Sign up at razorpay.com -> Dashboard -> Settings -> API Keys -> Generate Test Mode Keys. You get a Key ID and Key Secret for testing — no real money moves. For test card payments use card number 4111 1111 1111 1111, any future date, and any CVV. For UPI, use success@razorpay in the UPI ID field.

03 Step 1 — Creating a Razorpay Order (Server Side)

```
def initiate_payment(request, appointment_id):
```
views.py — triggered when patient clicks Pay Now
```
appointment = get_object_or_404(Appointment, id=appointment_id, patient=request.user)
```
Fetch and confirm ownership
```
    amount_paisa = int(appointment.fee * 100)
```
Razorpay uses paisa — Rs 500 = 50000 paisa
```
order = rz_client.order.create({'amount': amount_paisa, 'currency': 'INR',
'payment_capture': 1})
```
Create order — payment_capture=1 means auto-capture on success
```
return render(request, 'payment.html', {'order': order, 'rzp_key':
settings.RAZORPAY_KEY_ID, 'appointment': appointment})
```
Pass order data and key to template

04 Step 2 — Frontend Checkout (Razorpay JS)

```
<script src="https://checkout.razorpay.com/v1/checkout.js"></script>
```
payment.html — include Razorpay JS (no download needed)
```
var options = {key: '{{ rzp_key }}', amount: {{ order.amount }}, currency: 'INR',
name: 'Doctor Finder', order_id: '{{ order.id }}', callback_url: '/payment/callback/',
prefill: {email: '{{ request.user.email }}'}};
```
JS options — all values injected from Django context
```
var rzp = new Razorpay(options); document.getElementById('pay-btn').onclick =
function(){rzp.open()};
```
Open Razorpay checkout when patient clicks Pay

05 Step 3 — Verifying the Payment (Critical Security Step)

WARNING — Always Verify the Signature

Never mark an appointment as paid based only on a success callback from the browser. A malicious user can fake that callback. Always verify the HMAC-SHA256 signature server-side using razorpay_order_id + razorpay_payment_id signed with your secret key. The SDK does this in one line.

```
@csrf_exempt                                          views.py — Razorpay POSTs to this callback
def payment_callback(request):                        Called by Razorpay after payment attempt
params = {'razorpay_order_id': request.POST['razorpay_order_id'],
'razorpay_payment_id': request.POST['razorpay_payment_id'], 'razorpay_signature':
request.POST['razorpay_signature']}
    Extract the three required fields
    try:                                              Attempt signature verification
rz_client.utility.verify_payment_signature(params)
    Raises SignatureVerificationError if tampered
appointment.payment_status = 'paid'; appointment.save()
    Only update status after valid signature
return redirect('/booking/confirmed/')
    Show success page
except razorpay.errors.SignatureVerificationError:
    Signature mismatch — possible fraud
        return redirect('/payment/failed/')           Show failure page
```

CASE STUDY — Lybrate Doctor Booking Platform

Context — Lybrate is an Indian doctor consultation platform serving patients across Gujarat, Maharashtra, and Delhi since 2013.

Challenge — Lybrate needed a single payment integration supporting UPI, debit/credit cards, and net banking — all common in India — without building their own payment infrastructure.

What they did — Lybrate integrated Razorpay as their unified payment gateway. Their backend verifies payment signatures before confirming bookings, exactly as shown in this session.

Result — Lybrate reduced payment failures by routing all Indian payment methods through Razorpay's smart retry system for failed UPI transactions.

Takeaway — A working Razorpay integration with signature verification is a real portfolio credential. Every Indian health-tech startup uses this pattern.

Source: Razorpay Case Studies, razorpay.com/case-studies

Payment Method	Supported by Razorpay	Popularity in India 2024	Notes

Payment Method	Supported by Razorpay	Popularity in India 2024	Notes
UPI (GPay, PhonePe, Paytm)	Yes	Most popular — 46% of transactions	Near-instant settlement, preferred by students
Debit / Credit Card	Yes	Very common	Supports EMI for amounts above Rs 3,000
Net Banking	Yes	Preferred by older users	60+ Indian banks supported
Paytm Wallet	Yes	Declining since UPI	Still useful for small amounts

WHERE TO GO NEXT

Doctor Finder now has a complete pay-at-booking flow with server-side fraud protection.

Explore deeper — Razorpay developer docs at razorpay.com/docs/payments/payment-gateway/web-integration — especially webhook setup for asynchronous payment confirmation

Try on your own — Add a Payment History page where logged-in patients can see all past transactions using rz_client.payment.fetch(payment_id) for each stored payment ID

Connect the dots — Next session adds Google Maps so patients can see where each doctor's clinic is located before booking

Going further — Explore Razorpay Subscriptions for monthly consultation plans, and Razorpay Smart Collect for automated UPI mandate collection

AI PROMPT — ASK CHATGPT / GEMINI / CLAUDE

Act as a fintech developer. Explain the Razorpay three-step payment flow in simple Hinglish — from patient clicking Pay Rs 500 to Django marking the appointment as confirmed. Explain in one sentence why signature verification on the server is critical. Use Doctor Finder as the example. Keep it under 150 words.

SESSION
18

60 MIN · MAPS & GEOLOCATION

Maps & Geolocation API

Learning Objectives

- Add latitude and longitude fields to the Doctor model and store them via the Geocoding API
- Convert a clinic's text address into coordinates using Google's Geocoding API
- Embed an interactive Google Map in a Django template with doctor markers
- Implement distance-based doctor search using the Haversine formula
- Use the browser Geolocation API to get a patient's current location

01 Why Location Makes Doctor Finder Useful

A patient in Bopal, Ahmedabad searching for an orthopaedic doctor wants to know which clinic is 2 km away — not just a list of names. Google Maps API lets you add this capability to Doctor Finder with roughly 30 lines of JavaScript and one server-side utility function.

CONCEPT — Two APIs, One Flow

Geocoding API converts a text address (like 'SG Highway, Ahmedabad') into latitude and longitude numbers. Maps JavaScript API then uses those numbers to drop a pin on an interactive map in the browser. You geocode once when a doctor registers, store the coordinates in your database, and use them every time a patient searches.

02 Adding Location Fields to the Doctor Model

models.py — add three new fields to existing Doctor model

```
class Doctor(models.Model):
    address = models.TextField()
```

Full clinic address as free text

```
latitude = models.DecimalField(max_digits=9, decimal_places=6, null=True, blank=True)
```

6 decimal places gives ~0.1 m accuracy

```
longitude = models.DecimalField(max_digits=9, decimal_places=6, null=True, blank=True)
```

Store alongside latitude

```
python manage.py makemigrations && python manage.py migrate
```

Apply new fields to the database

03 Geocoding a Clinic Address

TRY THIS LIVE

Go to console.cloud.google.com -> Enable Maps JavaScript API and Geocoding API -> Create an API key -> Restrict it to your domain or IP. Add GOOGLE_MAPS_KEY to your .env file. Test by geocoding 'Navrangpura, Ahmedabad' and printing the returned lat/lng — it should be near 23.0393, 72.5527.

```
import requests                                          views.py — standard library, likely installed
GEOCODE_URL = 'https://maps.googleapis.com/maps/api/geocode/json'
    Google Geocoding endpoint
def geocode_address(address):                            Utility function — call when a doctor's address is saved
resp = requests.get(GEOCODE_URL, params={'address': address, 'key':
config('GOOGLE_MAPS_KEY')}).json()
    Call the API
    if resp['status'] == 'OK':                           Only proceed if geocoding succeeded
loc = resp['results'][0]['geometry']['location']
    Extract location dict {lat, lng}
        return loc['lat'], loc['lng']                    Return as a tuple
    return None, None                                    Return safe nulls on failure
```

◆ Call geocode_address from your save view

```
doctor.latitude, doctor.longitude = geocode_address(doctor.address)
    In your CreateDoctor or UpdateDoctor view, after form.save()
doctor.save()                                            Persist the coordinates — geocode again only if
                                                         address changes
```

04 Embedding the Map in a Template

```
<div id="map" style="height:450px;width:100%;"></div>
    doctors.html — the map container div
<script src="https://maps.googleapis.com/maps/api/js?key={{ google_key }}"></script>
    Load Maps JS — key passed from view context
const map = new google.maps.Map(document.getElementById('map'),
{center:{lat:23.0225,lng:72.5714}, zoom:12});
    Centre on Ahmedabad by default
const infoWindow = new google.maps.InfoWindow();
    Shared popup window for marker details
```

PRO TIP — Render Doctor Markers with Django Template Loop

Inside a script block in your template, loop over the doctors context variable using Django template tags and emit one new google.maps.Marker(...) call per doctor. Set the marker title to the doctor's name and attach an InfoWindow with their specialization and phone. This renders cleanly because Django processes the template server-side before the browser sees it.

05 Distance-Based Search — Haversine Formula

CONCEPT — Why Not Simple Subtraction?

At the scale of a city, latitude and longitude degrees are not equal distances. One degree of latitude is always ~111 km, but one degree of longitude varies from ~111 km at the equator to 0 km at the poles. In Ahmedabad (23 deg N), one longitude degree is about 102 km. The Haversine formula accounts for Earth's curvature and gives accurate kilometre distances between any two points.

```
import math                                                    views.py
def haversine(lat1, lon1, lat2, lon2):                         Returns distance in km between two lat/lng points
    R = 6371                                                   Earth's mean radius in km
dlat = math.radians(lat2 - lat1); dlon = math.radians(lon2 - lon1)
   Convert degree differences to radians
a = math.sin(dlat/2)**2 + math.cos(math.radians(lat1)) * math.cos(math.radians(lat2))
* math.sin(dlon/2)**2
   Haversine formula intermediate
    return R * 2 * math.asin(math.sqrt(a))                     Final distance in km
nearby_doctors = [d for d in Doctor.objects.exclude(latitude=None) if
haversine(patient_lat, patient_lng, float(d.latitude), float(d.longitude)) <= 5]
   Filter to doctors within 5 km of the patient
```

◆ Getting the Patient's Location (Browser Side)

```
navigator.geolocation.getCurrentPosition(function(pos){
   JavaScript — browser asks user for location permission
var lat = pos.coords.latitude; var lng = pos.coords.longitude;
   Coordinates from device GPS or network
window.location = '/doctors/nearby/?lat=' + lat + '&lng=' + lng;
   Redirect to nearby search view with coordinates as query params
});                                                            Close callback
```

Client-Side Geocoding

Browser calls the Geocoding API directly in JavaScript. Simpler to code, but your API key is visible in the HTML source — anyone can copy and misuse it.

Server-Side Geocoding

Your Django view calls the Geocoding API. The key stays on the server and is never sent to the browser. Always use server-side geocoding in production apps.

CASE STUDY — Ola's Nearest Driver Matching

Context — Ola operates across 250+ Indian cities including Ahmedabad, powering millions of daily rides.

Challenge — Ola must match a passenger's real-time location with the nearest available driver within seconds, at the scale of an entire city.

What they did — Ola's backend stores driver lat/lng (updated every few seconds) and runs geospatial proximity queries — conceptually the same Haversine-based radius search you built today, but at massive scale using PostGIS and Redis geospatial indexes.

Result — Average driver arrival time in Ahmedabad dropped under 5 minutes because matching finds the truly nearest driver rather than a random available one.

Takeaway — The distance search you built is the same core concept powering Ola, Zomato Delivery, and Practo's clinic finder. Scale and storage engine are the only differences.

Source: Ola Engineering Blog, engineering.olacabs.com

WHERE TO GO NEXT

Doctor Finder now shows clinic locations on a live map and can filter by distance from the patient.

Explore deeper — Google Maps Platform documentation at developers.google.com/maps/documentation — especially the Places API for adding address autocomplete to the doctor registration form

Try on your own — Add a Use My Location button on the search page that calls navigator.geolocation and re-runs the nearby doctors query automatically

Connect the dots — Next session deploys Doctor Finder live on PythonAnywhere so anyone on the internet can access it

Going further — Explore GeoDjango — Django's built-in GIS framework — and PostGIS for production-grade spatial queries with database-level distance filtering instead of Python loops

AI PROMPT — ASK CHATGPT / GEMINI / CLAUDE

Act as a geospatial developer. Explain the Geocoding API and the Haversine formula in simple Hinglish. Use the example of finding the nearest doctor in Ahmedabad. Explain in two sentences why we cannot just subtract latitude and longitude to get distance. Keep the total response under 150 words.

SESSION
19

60 MIN · DEPLOYMENT

GitHub Deployment & PythonAnywhere Hosting

Learning Objectives

- Push a Django project to GitHub with a proper .gitignore and requirements.txt
- Create a PythonAnywhere account and configure a Django web app
- Clone and run the project on PythonAnywhere via the Bash console
- Set production-safe Django settings: DEBUG=False, ALLOWED_HOSTS, and STATIC_ROOT
- Run collectstatic and database migrations on the production server

01 Deployment Is Part of the Job

A Django project that runs only on your laptop is not a product — it's a homework file. Every employer and client expects you to deploy. PythonAnywhere offers a free hosting tier built specifically for Django, and pairing it with GitHub gives you a professional deployment workflow in under an hour.

> **CONCEPT — Development vs Production**
>
> On your local machine, Django runs with DEBUG=True, serves your own static files, and uses SQLite. In production, DEBUG must be False (it exposes your entire codebase to errors in the browser), static files are served by a web server like Nginx, and you use MySQL. PythonAnywhere handles all of this once you configure three settings correctly.

02 Preparing Your Project for GitHub

```
pip freeze > requirements.txt                          Generate a list of all installed packages — run inside your virtualenv

echo '.env' > .gitignore                               First rule: never commit secrets
echo '__pycache__/' >> .gitignore                      Exclude Python bytecode cache
echo '*.sqlite3' >> .gitignore                         Exclude local database file
echo 'media/' >> .gitignore                            Exclude user-uploaded files (large, not code)
echo 'staticfiles/' >> .gitignore                      Exclude generated static root
git init                                               Initialise a Git repository in your project folder
git add .                                              Stage all files (excluding .gitignore entries)
git commit -m 'Initial Doctor Finder project commit'
   First commit
git remote add origin https://github.com/yourusername/doctor-finder.git
   Link to your GitHub repo
git push -u origin main                                Push to GitHub
```

PRO TIP — SECRET_KEY Must Never Be in Git

Django's SECRET_KEY is used to sign cookies and sessions. If it leaks on GitHub, anyone can forge session tokens and log in as any user on your site. Move it to your .env file before your very first git commit. On PythonAnywhere, you will set it as an environment variable in the web app dashboard.

03 Deploying on PythonAnywhere

TRY THIS LIVE

1) Sign up free at pythonanywhere.com. 2) Go to Dashboard -> Consoles -> Bash -> Start a new console. 3) Clone your repo: git clone https://github.com/yourusername/doctor-finder.git. 4) Create virtualenv: mkvirtualenv --python=python3.11 doctorenv. 5) Activate: workon doctorenv. 6) Install deps: pip install -r doctor-finder/requirements.txt. 7) Go to Web tab -> Add a new web app -> Manual configuration -> Python 3.11.

```
git clone https://github.com/yourusername/doctor-finder.git
```
PythonAnywhere Bash console — clone your project

```
mkvirtualenv --python=python3.11 doctorenv
```
Create a virtualenv named doctorenv

```
workon doctorenv
```
Activate the virtualenv

```
cd doctor-finder && pip install -r requirements.txt
```
Install all project dependencies

```
python manage.py migrate
```
Run all migrations on the production database

```
python manage.py createsuperuser
```
Create the admin account on production

```
python manage.py collectstatic
```
Copy all static files to STATIC_ROOT

04 Configuring the WSGI File

```
# In PythonAnywhere Web tab -> click the WSGI configuration file link
```
PythonAnywhere auto-creates this file

```
import sys, os
```
WSGI file — top of file

```
path = '/home/yourusername/doctor-finder'
```
Absolute path to your project root

```
if path not in sys.path: sys.path.append(path)
```
Add project to Python path

```
os.environ['DJANGO_SETTINGS_MODULE'] = 'doctorfinder.settings'
```
Tell Django which settings module to use

```
from django.core.wsgi import get_wsgi_application
```
Import WSGI handler

```
application = get_wsgi_application()
```
Expose the WSGI callable — PythonAnywhere calls this

05 Production Django Settings

WARNING — DEBUG=True in Production is Dangerous

Django's DEBUG=True mode shows a full traceback including your source code, installed packages, and environment variables when any error occurs. Any visitor who triggers a 500 error can read your entire codebase. Always set DEBUG=False before going live.

```
DEBUG = config('DEBUG', default=False, cast=bool)
    settings.py — read from env, default to False on production
ALLOWED_HOSTS = ['yourusername.pythonanywhere.com']
    Must match your PythonAnywhere domain exactly
SECRET_KEY = config('SECRET_KEY')                Read from env var set in PythonAnywhere dashboard
STATIC_ROOT = os.path.join(BASE_DIR, 'staticfiles')
    Where collectstatic copies all files to
STATIC_URL = '/static/'                          URL path for static files
MEDIA_ROOT = os.path.join(BASE_DIR, 'media')     Where uploaded files are stored on server
MEDIA_URL = '/media/'                            URL path for uploaded media files
```

Deployment Step	Command / Location	Common Mistake
Create requirements.txt	pip freeze > requirements.txt — run inside virtualenv	Forgetting to update it after adding new packages
Protect secrets	Add .env to .gitignore before first commit	Committing SECRET_KEY or database passwords
Clone on server	git clone in PythonAnywhere Bash console	Using HTTP clone without a personal access token on private repos
WSGI config	PythonAnywhere Web tab -> WSGI file	Wrong path or settings module name causes 500 errors
Static files	python manage.py collectstatic	Forgetting to run this — CSS and JS will not load in production
ALLOWED_HOSTS	Add PythonAnywhere domain in settings.py	Leaving it empty causes a 400 Bad Request on every page

CASE STUDY — Zerodha's Python-First Culture

Context — Zerodha, India's largest stockbroker by active clients, is headquartered in Bengaluru and serves millions of traders including tens of thousands from Gujarat.

Challenge — Zerodha's engineering team needed a deployment culture where every developer could ship and verify their own features quickly, without a dedicated DevOps team blocking releases.

What they did — Zerodha built a Python-first stack where all developers are expected to understand Git workflows, environment variable management, and basic server configuration — skills taught in this session.

Result — Zerodha's engineering team of fewer than 100 engineers serves millions of daily active users, a ratio far better than most Indian fintech firms, because every developer owns their code end-to-end.

Takeaway — Deployment is not a DevOps specialist's job in modern Indian startups — it's expected of every Python developer.

Source: Zerodha Tech Blog, zerodha.tech

WHERE TO GO NEXT

Doctor Finder is now live at yourusername.pythonanywhere.com — accessible to anyone in the world.

Explore deeper — PythonAnywhere help pages at help.pythonanywhere.com — especially the MySQL setup guide for switching from SQLite to a production database

Try on your own — Set up automatic deployment: add a git pull && pip install -r requirements.txt && python manage.py migrate script to PythonAnywhere, triggered via their API on every GitHub push

Connect the dots — Final session integrates all features and prepares the project for your portfolio and GitHub presentation

Going further — Explore deployment on AWS EC2 — the approach used by most Indian startups — using Gunicorn, Nginx, and Supervisor for a production-grade Django stack

AI PROMPT — ASK CHATGPT / GEMINI / CLAUDE

Act as a Django DevOps engineer. Explain in simple Hinglish why we need to set DEBUG=False, configure ALLOWED_HOSTS, and run collectstatic before deploying a Django project. Use a real-world analogy — like the difference between a kitchen in a cooking class versus a restaurant kitchen. Keep it under 150 words.

SESSION
20

60 MIN · CAPSTONE PROJECT

Capstone — Doctor Finder: Integration, Polish & Portfolio

Learning Objectives

- Verify that all Doctor Finder features work together end-to-end in production
- Perform structured testing across every major user journey
- Clean up code quality issues before the final GitHub push
- Write a professional README that presents the project to employers
- Articulate what you built and why each technical decision was made — for interviews

01 What You Have Built

Over 20 sessions you have built Doctor Finder — a full-stack health-tech web application that mirrors how real platforms like Practo and Lybrate operate. It handles user accounts, real payments, live maps, email and SMS, file uploads, role-based access, and is running on a public URL. This is not a tutorial exercise — it is a deployable product.

PYTHON DEVELOPER DEMAND IN INDIA

Rs 6–12 LPA

Source: Glassdoor India 2024 — typical starting salary range for Django / Python developer roles in Ahmedabad, Surat, and Pune

02 Full Feature Integration Checklist

Feature	Sessions Covered	What to Verify
User Registration + Login (email)	08	New user can register, log in, and log out
Google OAuth2 Social Login	16	Clicking Sign in with Google creates or fetches a user and redirects to dashboard
SMS OTP Phone Verification	16	OTP arrives within 30 seconds, expired OTP is rejected
Doctor Profile CRUD	12	Admin can add, view, edit, and delete a doctor

Feature	Sessions Covered	What to Verify
Profile Photo Upload	11	Doctor photo uploads, displays on profile page, serves from /media/
Email Confirmation on Booking	09 + 16	Patient receives email with appointment details after booking
Appointment Booking	12	Patient can book a slot with a doctor and see it in their history
Payment via Razorpay	17	Test card completes payment, signature is verified, status shows confirmed
Google Maps — Clinic Marker	18	Each doctor's clinic appears as a pin on the map
Distance-Based Search	18	Searching nearby returns only doctors within the set radius
Password Reset Flow	10	Forgot password -> OTP link -> new password works
Role-Based Access	15	Patients cannot access doctor management pages; doctors cannot access admin
Django Admin Panel	05	Admin can manage all models with filters and search
Live Deployment	19	App loads at PythonAnywhere URL with HTTPS, no 500 errors

03 Testing Every User Journey

TRY THIS LIVE — Three Journeys to Test

Patient Journey: Open the site in Incognito -> register with Google -> search for a cardiologist near your location -> book an appointment -> complete a test payment -> check your email for confirmation. | Doctor Journey: Log in as a doctor -> update your profile photo -> view your appointment list -> check a confirmed booking. | Admin Journey: Open /admin -> add a new specialization -> add a test doctor -> verify they appear on the public map and search results.

04 Code Cleanup Before Final Push

```
grep -rn 'print(' .
```
Find all print() debug statements in your project — remove them all

```
grep -rn 'DEBUG = True' .
```
Verify no file has DEBUG hardcoded as True

```
pip freeze > requirements.txt
```
Regenerate requirements.txt — ensure it reflects your final package list

```
python manage.py check --deploy
```
Django's built-in deployment checklist — fix every warning it raises

```
python manage.py test
```
Run any tests you have written — all should pass before pushing

```
git add . && git commit -m 'Final capstone: Doctor Finder v1.0 — all features integrated'
```
Final commit with a meaningful message

```
git push origin main
```
Push to GitHub

```
# On PythonAnywhere console:
```
Deploy the final version to production

```
cd doctor-finder && git pull && pip install -r requirements.txt && python manage.py migrate && python manage.py collectstatic --noinput
```
Pull latest code and refresh production

PRO TIP — python manage.py check --deploy

Django's deployment checker tests your settings against a list of known security misconfigurations. It will warn you about missing HTTPS settings, a weak SECRET_KEY, sessions not marked as secure, and more. Fix every warning before sharing your URL with an employer — these are the same checks a senior developer does before going live.

05 Writing Your GitHub README

```
# Doctor Finder
```
README.md — your project name as H1

```
> A full-stack Django health-tech web application for discovering and booking doctor appointments.
```
One-line description

```
## Live Demo
```
Paste your PythonAnywhere URL here — this is what employers click first

```
## Features
```
Bullet list: Google Login, SMS OTP, Razorpay Payment, Google Maps, Role-Based Access, Email Notifications

```
## Tech Stack
```
Django 4.x | Python 3.11 | MySQL | Razorpay | Twilio | Google Maps API | PythonAnywhere

`## Setup Instructions`	git clone -> create .env -> pip install -r requirements.txt -> migrate -> runserver
`## Screenshots`	Add 3-4 screenshots: home page, map view, payment page, admin panel

06 Presenting This Project in an Interview

PRO TIP — How to Talk About This Project

Do not say: 'I made a doctor booking website.' Say: 'I built a full-stack Django application with Google OAuth2 for social login, Razorpay payment integration with server-side HMAC signature verification, a geolocation-based doctor search using the Haversine formula, and SMS OTP verification via Twilio. The project is deployed on PythonAnywhere with production-safe settings.' Specific technology names and specific decisions — that is what interviewers listen for.

Interview Question	What to Say
Why Razorpay over PayPal?	Razorpay supports UPI, the most popular payment method in India, with no monthly fee on the free tier
How did you prevent payment fraud?	I verify the HMAC-SHA256 payment signature server-side before updating the appointment status
How does the map search work?	I geocode clinic addresses once on save, store lat/lng, then use the Haversine formula to filter doctors by radius
How did you manage secret keys?	python-decouple reads from a .env file locally; environment variables set in PythonAnywhere dashboard for production
What would you improve?	Switch the Haversine Python loop to a PostGIS database-level spatial query for better performance at scale

“A deployed project with a live URL and a clean README tells an employer more than ten certificates.”

WHERE TO GO NEXT

Doctor Finder is complete and live. You have built something real.

Explore deeper — Django REST Framework (Module 5 of this course) — convert Doctor Finder's views into

a REST API so a React or Flutter frontend can consume it

Try on your own — Share your PythonAnywhere URL with a friend or family member — watch them use it as a real user and note every point of confusion

Connect the dots — Module 5 starts with DRF, which will let you separate Doctor Finder into a backend API and a modern frontend

Going further — Explore deployment on AWS EC2 with Gunicorn, Nginx, and GitHub Actions CI/CD — the production stack used by most Indian tech startups

AI PROMPT — ASK CHATGPT / GEMINI / CLAUDE

Act as a senior Django developer reviewing a student's project. I have built a Doctor Finder web app in Django with Google OAuth2, Razorpay payments with signature verification, Twilio SMS OTP, Google Maps geolocation search, and deployment on PythonAnywhere. Give me five specific interview questions a recruiter at an Indian tech company would ask about this project, and a strong one-sentence answer for each. Keep the total under 200 words.

REVIEW
?

INTERVIEW PREP · MODULE 4

Interview Questions

◆ DB and Python Framework (Django)

Rehearse each answer out loud and aim to deliver it within 60 seconds. The hint is only a nudge — attempt the answer first, then compare with the model answer below.

1

What is Django and why is it popular for web development?

Think about rapid development and built-in features.

MODEL ANSWER

Django is a high-level Python web framework that follows the 'batteries included' philosophy — it comes with a built-in admin panel, ORM, authentication system, form handling, and much more out of the box. It is popular because it lets developers build full-featured web applications rapidly without reinventing common components. Its large community, excellent documentation, and emphasis on security make it a reliable choice for production projects.

2

Explain the main differences between Django and Flask.

Focus on project structure and how much is provided out-of-the-box.

MODEL ANSWER

Django is a full-stack framework with a strict project structure and many built-in features — ORM, admin, auth, forms. Flask is a microframework that gives you almost nothing by default, letting you pick and add only the libraries you need. Django is better for larger, feature-rich applications; Flask suits small APIs or projects where you want full control over every component.

3

What is the MVT architecture in Django?

Describe the roles of Model, View, and Template.

MODEL ANSWER

MVT stands for Model-View-Template. The Model handles data — it defines database structure and contains business logic for accessing data. The View processes requests, queries models, and decides what response to return. The Template is the HTML layer that receives data from the view and renders it for the browser. Django's URL dispatcher connects incoming requests to the correct view.

4

How do you install Django and start a new project?

Mention the command-line tools involved.

MODEL ANSWER

Install Django with pip install django. Then create a new project using django-admin startproject projectname, which creates a directory with the project settings and configuration files. Navigate into that directory and run python manage.py runserver to start the development server and verify everything is working on localhost:8000.

5 **After creating a new Django project, what are the main files and folders you see in the project structure?**

MODEL ANSWER
You see a manage.py file at the root — used to run commands. Inside the project folder you find settings.py (all configuration), urls.py (root URL routing), wsgi.py and asgi.py (deployment entry points), and __init__.py (marks it as a Python package). This outer project folder is distinct from apps you create inside it.

6 **What is the difference between a Django project and a Django app?**

One is the whole site, the other is a reusable component.

MODEL ANSWER
A project is the entire web application — it contains global settings, root URL configuration, and one or more apps. An app is a self-contained module within the project that handles a specific feature, such as 'blog', 'accounts', or 'shop'. One project can contain many apps, and an app can in principle be reused across different projects.

7 **How do you map a URL to a view in Django?**

Think about the urls.py file.

MODEL ANSWER
In urls.py you add a path() entry to urlpatterns that links a URL string to a view function. For example: path('home/', views.home, name='home'). When a request comes in for /home/, Django finds this entry and calls the home view function. For apps, you include their URL file in the project's root urls.py using include().

8 **What is the purpose of the settings.py file in a Django project?**

MODEL ANSWER
settings.py is the central configuration file for the entire project. It controls the database connection, installed apps, middleware, static and media file paths, email settings, secret key, debug mode, allowed hosts, and much more. Changing settings here affects the entire project — so production settings should always have DEBUG=False and ALLOWED_HOSTS set correctly.

9 **Write a simple Django view that returns 'Hello, World!' as an HttpResponse.**

MODEL ANSWER
```
from django.http import HttpResponse

def hello(request):
return HttpResponse('Hello, World!')
```

Every view function takes a request object as its first argument and must return an HttpResponse. Map this view in urls.py with path('hello/', views.hello) to make it accessible at /hello/.

10 **How would you create a new Django app called 'accounts' and register it with your project?**

MODEL ANSWER
Run python manage.py startapp accounts to create the app folder with its default files. Then open settings.py and add 'accounts' to the INSTALLED_APPS list. Without this registration, Django does not process the app's models, migrations, or template directories.

11 **What is the purpose of the templates folder in a Django app?**

MODEL ANSWER
The templates folder stores HTML files that Django's template engine uses to render responses. Views pass context data (variables) to a template, and the template engine replaces placeholders like {{ variable }} with real values. Keeping HTML in templates separates presentation from business logic, following the MVT pattern.

12 **Explain template inheritance in Django and its benefits.**

Think about avoiding repetition across pages.

MODEL ANSWER
Template inheritance lets you define a base template (base.html) with the common layout — navigation, header, footer — and use {% block content %}{% endblock %} placeholders. Child templates extend the base with {% extends 'base.html' %} and fill in those blocks. This eliminates repetition: updating the navigation in one base file updates every page instantly.

13 **How do you load static files like CSS or JavaScript in a Django template?**

MODEL ANSWER
At the top of the template add {% load static %}, then reference files with {% static 'path/to/file.css' %}. For example: <link rel='stylesheet' href='{% static "css/style.css" %}'>. Django resolves this to the correct URL based on STATIC_URL in settings. You also need 'django.contrib.staticfiles' in INSTALLED_APPS.

14 **How would you add Bootstrap to your Django project using a CDN?**

MODEL ANSWER
In your base.html template, paste the Bootstrap CDN link tags directly in the <head> section — no Django static file setup needed for CDN. For example, add the Bootstrap CSS link and the JS bundle script tag from getbootstrap.com. All child templates extending base.html will automatically have Bootstrap available.

15 **What is a Django model and why do we use it?**

Consider how Django interacts with the database.

MODEL ANSWER
A Django model is a Python class that maps directly to a database table. Each class attribute represents a column with its data type and constraints. Django's ORM translates Python operations on model objects into SQL automatically, so you rarely need to write raw SQL. This makes database interactions portable, readable, and less error-prone.

16 **How do you create a model for a 'Doctor' with fields for name and specialization?**

MODEL ANSWER

```
from django.db import models

class Doctor(models.Model):
name = models.CharField(max_length=100)
specialization = models.CharField(max_length=100)

def __str__(self):
return self.name
```

Place this in models.py inside your app. After creating or modifying models you must run makemigrations and migrate to update the database.

17 **What is the process to apply changes to your Django models in the database?**

Mention the commands used to create and apply migrations.

MODEL ANSWER

Run python manage.py makemigrations to generate migration files describing the changes to your models. Then run python manage.py migrate to apply those changes to the actual database. Migrations track schema history, allow rollbacks, and make it safe to evolve the database schema over time without losing data.

18 **How would you connect a Django project to a MySQL database instead of the default SQLite?**

MODEL ANSWER

Install mysqlclient (pip install mysqlclient), then update the DATABASES setting in settings.py: set ENGINE to 'django.db.backends.mysql' and provide NAME, USER, PASSWORD, HOST, and PORT. Run python manage.py migrate afterwards to create Django's system tables in MySQL. Make sure the MySQL database and user exist before running migrations.

19 **What steps are needed to make a new model appear in the Django admin interface?**

MODEL ANSWER

Open admin.py in your app and register the model: from .models import Doctor; admin.site.register(Doctor). Also create a superuser with python manage.py createsuperuser. Navigate to /admin/ in the browser, log in, and the Doctor model will appear with full CRUD functionality automatically generated.

20 **How can you customize the admin list display for a model in Django?**

MODEL ANSWER

Create a ModelAdmin class with list_display set to the fields you want as columns: class DoctorAdmin(admin.ModelAdmin): list_display = ['name', 'specialization']. Register it with admin.site.register(Doctor, DoctorAdmin). You can also add list_filter, search_fields, and ordering to make the admin list more useful.

21 **How would you add a filter for specialization in the Django admin for the Doctor model?**

MODEL ANSWER
In your ModelAdmin class, add list_filter = ['specialization']. Django renders a sidebar filter panel in the admin list view with all unique specialization values as clickable options. Clicking a value filters the list to show only doctors with that specialization — no extra code needed.

22 **What is the difference between a Django Form and a ModelForm?**

Think about how each form is created and what it is used for.

MODEL ANSWER
A Form is a standalone form class where you manually define each field. A ModelForm is tied directly to a model — it automatically generates form fields from the model's fields, and its save() method writes validated data straight to the database. Use ModelForm when your form maps closely to a model; use a plain Form for custom logic not tied to a single model.

23 **Explain the difference between GET and POST requests in the context of Django forms.**

MODEL ANSWER
GET requests send data in the URL query string and are used for fetching data — search forms, filtering, and navigation. POST requests send data in the request body, making them suitable for creating or modifying data since the payload is not visible in the URL. In Django, form submissions that change the database should always use POST, and you validate with if request.method == 'POST'.

24 **Why is the CSRF token important when handling form submissions in Django?**

Consider security for web forms.

MODEL ANSWER
CSRF (Cross-Site Request Forgery) protection prevents a malicious website from tricking a logged-in user's browser into submitting a form to your site. Django generates a unique token per session and embeds it as a hidden field with {% csrf_token %}. On POST, Django checks the token — if it does not match, the request is rejected. Without it, attackers could submit unauthorised actions on behalf of your users.

25 **Write a Django Form class to add a new Doctor with fields for name and phone number.**

MODEL ANSWER
from django import forms

class DoctorForm(forms.Form):
name = forms.CharField(max_length=100)
phone = forms.CharField(max_length=15)

This form can be instantiated in a view with form = DoctorForm(request.POST) and validated with form.is_valid(). Accessing form.cleaned_data['name'] gives you the sanitised value.

26 **How would you display validation errors in a Django form template?**

MODEL ANSWER

Pass the form to the template as context and use {{ form.field.errors }} to show errors next to each field, or {{ form.errors }} to show all errors at the top. A cleaner option is to render the whole form with {{ form.as_p }} — Django automatically includes error messages next to any invalid field. Always re-render the form with errors when validation fails, not just redirect.

27 **How does a ModelForm help in saving data to the database?**

Think about automatic field mapping.

MODEL ANSWER

A ModelForm's Meta class specifies the model and which fields to include. When you call form.save() after validation, Django automatically creates or updates the corresponding database record — you do not need to manually extract each field and call Model.objects.create(). This eliminates boilerplate and keeps form and model in sync when fields change.

28 **Write a view function that uses a ModelForm to save a new Doctor to the database.**

MODEL ANSWER

```
def add_doctor(request):
if request.method == 'POST':
form = DoctorForm(request.POST)
if form.is_valid():
form.save()
return redirect('doctor_list')
else:
form = DoctorForm()
return render(request, 'add_doctor.html', {'form': form})
```

On GET the empty form is shown; on POST the data is validated and saved. Redirecting after a successful save prevents duplicate submissions on page refresh.

29 **How does Django handle user authentication (signup, login, logout) by default?**

MODEL ANSWER

Django's django.contrib.auth app provides a ready-made User model, authentication views, and utility functions. authenticate() verifies credentials, login() attaches the user to the session, and logout() clears it. For signup you typically create a form based on UserCreationForm. Django also provides built-in URL patterns for login and logout at django.contrib.auth.urls.

30 **What is password hashing and why is it important in user authentication?**

MODEL ANSWER
Password hashing transforms a plain-text password into a fixed-length scrambled string using an algorithm like PBKDF2 or bcrypt. The hash is stored, not the original password, so even if the database is compromised attackers cannot recover user passwords. Django handles this automatically through its authentication system — you never store or compare plain-text passwords directly.

31 **How would you display the logged-in user's username in the navigation bar of your Django site?**

MODEL ANSWER
Django makes the request object available in templates via the context processor django.template.context_processors.request. In any template you can access the logged-in user as {{ request.user.username }}. If the user is not logged in, request.user is an AnonymousUser, so wrap it with {% if request.user.is_authenticated %} to show it only when logged in.

32 **What steps are required to send an email in Django upon user signup?**

MODEL ANSWER
Configure email settings in settings.py: set EMAIL_BACKEND, EMAIL_HOST, EMAIL_PORT, EMAIL_HOST_USER, and EMAIL_HOST_PASSWORD. In your signup view, after saving the user call send_mail(subject, message, from_email, [to_email]). For a real SMTP service like Gmail or SendGrid, use the corresponding backend and credentials. For development, EMAIL_BACKEND = 'django.core.mail.backends.console.EmailBackend' prints emails to the terminal.

33 **How can you use HTML email templates when sending emails in Django?**

MODEL ANSWER
Use EmailMultiAlternatives instead of send_mail. Render an HTML template to a string with render_to_string('email_template.html', context), then attach it with msg.attach_alternative(html_content, 'text/html'). Always include a plain-text fallback as the main body, as some email clients do not render HTML. This approach separates email content from code.

34 **Describe how Django sessions work and what they are used for.**

Think about storing user data between requests.

MODEL ANSWER
HTTP is stateless, so Django sessions store user-specific data on the server between requests. When a session is created, Django sends the client a session ID cookie. On subsequent requests, Django looks up the server-side session data using that ID. Sessions are used to maintain login state, store shopping cart data, pass flash messages, and temporarily hold OTPs.

35 **How would you implement a forgot password feature using OTP and Django sessions?**

MODEL ANSWER
Generate a random OTP, send it to the user's email, and store it in the session: request.session['otp'] = otp. On the verification page, compare the user's input against request.session['otp']. If they match, redirect to a reset password page; otherwise show an error. After successful verification, delete the OTP from the session and allow the user to set a new password.

36 **What are MEDIA_ROOT and MEDIA_URL settings used for in Django?**

MODEL ANSWER
MEDIA_ROOT is the absolute filesystem path where Django saves files uploaded by users. MEDIA_URL is the public URL prefix that serves those uploaded files. For example, MEDIA_ROOT = BASE_DIR / 'media' and MEDIA_URL = '/media/'. In development you also need to add a URL pattern in urls.py to serve media files; in production a web server like Nginx serves them directly.

37 **How would you allow users to upload and update their profile pictures in a Django app?**

MODEL ANSWER
Add an ImageField to the model (requires Pillow installed). In the ModelForm include that field and set the form's enctype to multipart/form-data. In the view, pass request.FILES alongside request.POST when instantiating the form: DoctorForm(request.POST, request.FILES, instance=obj). Calling form.save() handles storing the file to MEDIA_ROOT and saving the path in the database.

38 **Explain the steps to create basic CRUD operations (Add, View, Edit, Delete) for the Doctor model.**

MODEL ANSWER
Create: a view with a ModelForm that saves on POST. Read: a list view using Doctor.objects.all() and a detail view using get_object_or_404(Doctor, pk=pk). Update: the same ModelForm view but initialised with instance=doctor to pre-fill values. Delete: a view that calls doctor.delete() on POST and redirects. Map each to a URL and link them from templates with {% url %} tags.

39 **How would you implement a delete confirmation page before removing a Doctor record?**

MODEL ANSWER
Create a delete view that on GET renders a confirmation template with the doctor's name. The template shows a form asking 'Are you sure?' with a POST button. On POST the view calls doctor.delete() and redirects to the list. This two-step pattern prevents accidental deletion from a direct GET request, which is also a security best practice.

40 **What is AJAX and how can it be used to delete a Doctor record in Django without refreshing the page?**

MODEL ANSWER
AJAX (Asynchronous JavaScript and XML) allows the browser to send HTTP requests and handle responses in the background without a full page reload. Use JavaScript's fetch() to send a POST request to your Django delete URL with the CSRF token in the headers. The Django view deletes the record and returns a JSON response; the JavaScript then removes the row from the DOM using the response.

41 **How would you send a JSON response from a Django view?**

MODEL ANSWER
```
from django.http import JsonResponse

def my_view(request):
data = {'status': 'success', 'message': 'Done'}
return JsonResponse(data)
```

JsonResponse automatically sets the Content-Type header to application/json and serialises the dictionary. Pass safe=False if you need to return a list rather than a dictionary.

42 **Explain the use of filter(), exclude(), and annotate() methods in Django ORM.**

MODEL ANSWER
filter() returns a QuerySet of objects matching the given conditions — Doctor.objects.filter(specialization='Cardiology'). exclude() returns objects that do NOT match the condition. annotate() adds computed fields to each object in the QuerySet, such as counting related appointments per doctor using Count(). They can be chained together for complex queries.

43 **What is the difference between select_related() and prefetch_related() in Django ORM?**

Both are for optimizing database queries with related objects.

MODEL ANSWER
select_related() performs a SQL JOIN and fetches related objects in a single query — best for ForeignKey and OneToOne relationships. prefetch_related() runs separate queries for each related table and joins them in Python — best for ManyToMany and reverse ForeignKey relationships. Both reduce the N+1 query problem where accessing a related field in a loop triggers a query for every row.

44 **How would you paginate a list of doctors in Django?**

MODEL ANSWER

```
from django.core.paginator import Paginator

doctors = Doctor.objects.all()
paginator = Paginator(doctors, 10)  # 10 per page
page_number = request.GET.get('page')
page_obj = paginator.get_page(page_number)
```

Pass page_obj to the template and use its attributes — page_obj.has_next(), page_obj.previous_page_number() — to build pagination links.

45 **How does the login_required decorator work in Django?**

MODEL ANSWER

Adding @login_required above a view function checks whether the requesting user is authenticated. If they are, the view runs normally. If not, Django redirects them to the login page (settings.LOGIN_URL, defaulting to /accounts/login/), appending the original URL as a next parameter so users are returned there after logging in.

46 **Describe how you would implement role-based access control, such as separate dashboards for doctors and patients.**

MODEL ANSWER

Add a role field to your user profile model (e.g., 'doctor' or 'patient'). In views, after authentication check the user's role with request.user.profile.role and redirect accordingly. Use a custom decorator or mixin to protect role-specific views. You can also use Django's Groups and Permissions system to assign permissions to each role and check them with user.has_perm().

47 **How can you restrict access to certain admin pages based on user permissions or groups?**

MODEL ANSWER

In Django admin you can override the has_module_perms() and has_change_permission() methods in a ModelAdmin subclass to return False for users who lack the required permissions or group membership. Alternatively, use Django's built-in permissions — assign view, add, change, delete permissions to groups in the admin, then add users to those groups.

48 **How would you integrate Google Login (social login) into a Django project?**

MODEL ANSWER

Install django-allauth and add it to INSTALLED_APPS along with 'allauth.socialaccount.providers.google'. Configure SITE_ID and AUTHENTICATION_BACKENDS in settings. Add the allauth URLs to urls.py. In the Google Cloud Console, create OAuth credentials with your callback URL and add the client ID and secret to the Django admin under Social Applications. Users can then log in with their Google account.

49 **Describe the steps to send an OTP via SMS using an API like Twilio in Django.**

MODEL ANSWER
Install the twilio library and store your Account SID, Auth Token, and Twilio phone number in settings or environment variables. In your view, generate a random OTP, store it in the session, then call twilio_client.messages.create(body=f'Your OTP is {otp}', from_=TWILIO_NUMBER, to=user_phone). On the verification form, compare the submitted OTP with the session value.

50 **What are the main steps to integrate a payment gateway (such as Stripe or Paytm) in a Django application?**

MODEL ANSWER
Install the gateway's Python SDK. Create a payment initiation view that calls the gateway's API with the order amount and returns a payment URL or token to the frontend. Redirect the user to the gateway's checkout page. After payment, the gateway redirects back to your callback URL. Verify the payment signature or status in the callback view before updating the appointment record as paid.

51 **How would you handle the payment callback and update the appointment status after a successful transaction?**

MODEL ANSWER
The gateway sends a POST request to your callback URL with transaction details. Verify the payment signature using the gateway's SDK to confirm the transaction is genuine. If valid, update the relevant appointment record with payment_status='paid' and save it. Return a success response to the gateway and redirect the user to a confirmation page.

52 **Explain how to embed a Google Map showing a doctor's clinic location in a Django template.**

MODEL ANSWER
Get a Google Maps Embed API key from Google Cloud Console. In your template, include an <iframe> with the Maps Embed API URL, passing the doctor's address or coordinates as query parameters. Alternatively, use the Google Maps JavaScript API to initialise a map in a and place a marker at the clinic's latitude and longitude, which you pass from the view as template context.

53 **How would you search for doctors by distance using geolocation APIs in Django?**

MODEL ANSWER
Use the Google Maps Geocoding API to convert the user's entered location to latitude and longitude. Then use the Haversine formula (or a library like geopy) to calculate distances between that point and each doctor's stored coordinates. Filter and sort the results by distance in Python and return the nearest doctors. For large datasets, consider using PostGIS or MySQL spatial extensions for database-level distance queries.

54 **What steps are required to deploy a Django project to PythonAnywhere and make it live?**

Think about code upload, static/media files, and database migration.

MODEL ANSWER
Push your project to GitHub, then pull it onto PythonAnywhere via Bash console. Create a new Web App pointing to your project directory and WSGI file. Configure the virtualenv path and install dependencies. Set DEBUG=False and update ALLOWED_HOSTS with your PythonAnywhere domain. Run migrations to set up the database, collectstatic for static files, and reload the web app.

55 **How do you configure static and media files for production deployment in Django?**

MODEL ANSWER
Set STATIC_ROOT to a directory (e.g., BASE_DIR / 'staticfiles') and run python manage.py collectstatic to gather all static files there. Set MEDIA_ROOT similarly for uploads. In production, configure your web server (Nginx or Apache) to serve both directories directly, bypassing Django for better performance. In PythonAnywhere, set the static files mapping in the web app configuration.

56 **Describe the process of pushing your Django project to GitHub and why version control is important.**

MODEL ANSWER
Initialise a git repo with git init, add a .gitignore to exclude venv, __pycache__, and .env files, then commit: git add . && git commit -m 'Initial commit'. Create a repo on GitHub and push with git remote add origin <url> && git push. Version control lets you track history, roll back bugs, collaborate with a team, and deploy confidently from a known state.

57 **How would you ensure your deployed Django site is working correctly after deployment?**

MODEL ANSWER
Visit each key URL and test core user flows — registration, login, a search, an appointment booking. Check the error logs on PythonAnywhere for any 500 errors. Verify static files load (CSS/JS visible), media uploads work, email sending functions, and the database has the correct data. Set up a health-check endpoint that returns 200 OK to monitor the site automatically.

58 **In your Doctor Finder project, how did you implement appointment booking and payment integration?**

Describe the workflow from user action to payment confirmation.

MODEL ANSWER
The user selects a doctor and an available slot, which creates a pending Appointment record. They are then redirected to a payment initiation view that calls the payment gateway API. On successful payment the gateway redirects to our callback URL; the view verifies the transaction signature, marks the appointment as confirmed, and sends a confirmation email. The doctor's dashboard then shows the confirmed appointment.

59 **How did you use filters such as specialization and location in your doctor search feature?**

MODEL ANSWER
The search form sends specialization and city as GET parameters. The view reads these with request.GET.get() and applies them as ORM filters — Doctor.objects.filter(specialization=spec, city=city) — combining them with Q objects or chained filters when multiple criteria are present. An empty filter returns all doctors. Results are passed to the template with pagination.

60 **Describe how you handled user registration, OTP verification, and password reset in your capstone project.**

MODEL ANSWER
On registration, the user submits name, email, and password. The view creates the user with is_active=False, generates an OTP, stores it in the session, and emails it. On the verify page, the submitted OTP is compared to the session value; if correct, is_active is set to True. For password reset, the same OTP flow is used but instead of activation it redirects to a set-new-password form.

61 **If you were to improve your Doctor Finder project, what features or optimizations would you add?**

MODEL ANSWER
I would add real-time availability using WebSockets so slot statuses update live, implement ElasticSearch for faster full-text doctor search, use Redis for caching popular specialization queries, and add a review and rating system for doctors. On the infrastructure side, I would move to a cloud server, configure Celery for async email/SMS tasks, and add comprehensive automated tests.

62 **How could you use AI tools like ChatGPT or GitHub Copilot to speed up Django development, and what would you check before using their code?**

Think about code suggestions and the importance of reviewing for security or correctness.

MODEL ANSWER
AI tools can generate boilerplate views, serialisers, URL patterns, and test cases very quickly. Copilot is especially useful inside the editor for completing repetitive patterns like ModelAdmin classes or migration stubs. Before using any suggestion you should verify it does not expose sensitive data, check that ORM queries are parameterised, and run your test suite. AI-generated Django code sometimes uses deprecated patterns, so always cross-reference with the official docs.

63 **How might you use AI to help with tasks like writing email templates or generating test data in a Django project?**

MODEL ANSWER
Ask ChatGPT to draft HTML email templates given your subject and variables — it can produce a polished template faster than writing it from scratch. For test data, prompt it to generate a list of realistic doctor names, specialisations, and phone numbers as Python fixture data or a management command. Always review generated fixtures for data consistency and privacy (avoid real personal information).